

TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 3 — Foundation Architectures

Every pattern this course has taught, drawn with real AWS icons

35 LESSONS

AUTHOR

Surender Sara

COMPANY

Northbay Solutions

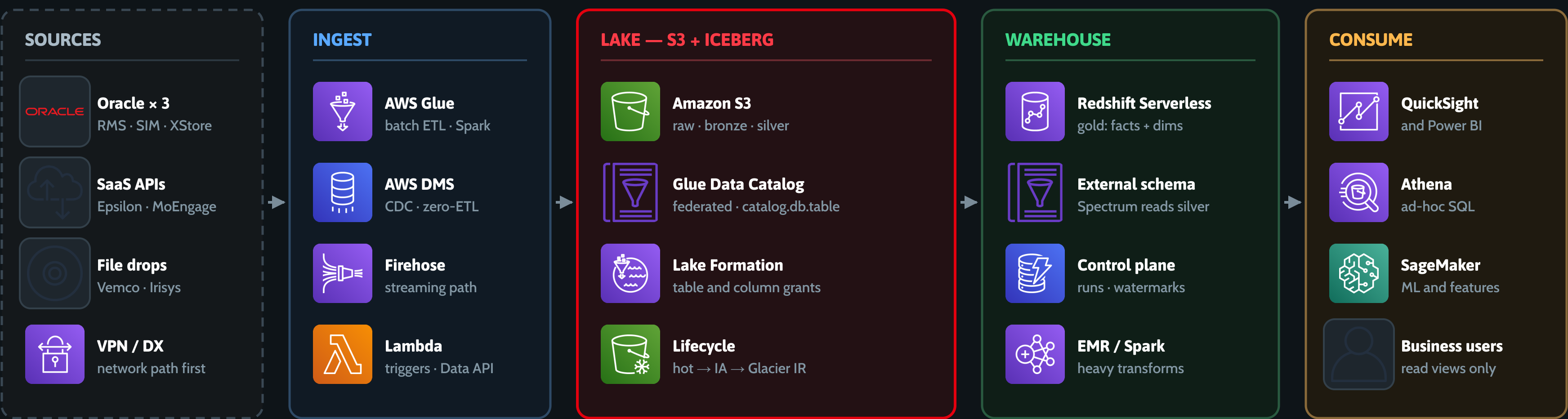
ISSUED

August 2026

The Lakehouse Reference Architecture

The pattern every module has been describing, drawn with the real AWS service icons you will use in your own diagrams.

1 Â· THE DATA PATH — LEFT TO RIGHT



UNDERNEATH ALL FIVE COLUMNS, ALL OF THE TIME

Lake Formation
who sees what

Glue Data Catalog
federated, three-level

Step Functions
orchestration

EventBridge
schedule and events

CloudWatch
metrics and alarms

IN PLAIN ENGLISH

Five stops on one road: arrive, unpack, store, model, sell. Everything in the strip below runs under all five, all of the time.

D01 · The Lakehouse Reference Architecture

Diagram: [_render/D01-lakehouse-reference.html](#)

What it shows

The five-stop road that every other diagram in this module sits inside: **sources** → **ingest** → **lake** → **warehouse** → **consume**, with a cross-cutting band underneath that runs beneath all five, all of the time.

If you only ever draw one diagram on a whiteboard, draw this one.

The five columns

Sources. On-prem databases, SaaS APIs and file drops — five different *classes*, not eight different problems (D19 in Module 0, D16 here). On-prem access arrives over VPN or Direct Connect, which is procurement with weeks of lead time, not a coding task.

Ingest. Glue for batch, DMS for CDC, zero-ETL where it fits, Transfer Family for files, Firehose if anything ever streams. One connector class per source class; one spec per table.

Lake — S3 + Iceberg. *raw/* lands immutable and is never edited. *bronze/* is typed and deduplicated. *silver/* is conformed and joined. All on S3 Tables, all MERGE-loaded on a key, all governed through the catalog. **One copy of the data.**

Warehouse. Redshift Serverless holds *gold/* — modelled facts and dimensions built by dbt. Spectrum reads silver *in place* rather than loading it. The control plane records what happened.

Consume. Power BI and QuickSight read reporting views, never base tables. Athena is for the questions nobody anticipated. *UNLOAD* publishes gold back to S3 so cheaper engines can read it without touching the warehouse.

The band underneath

LAYER	SERVICE
Catalog	Glue Data Catalog, federated (D06)
Access control	Lake Formation — catalog / database / table / column
Orchestration	Step Functions + EventBridge (D25)
Observability	CloudWatch (D26)
All of it, as code	Terraform (D13)

These are not a footnote. They are what makes five columns a platform rather than five projects sharing a bucket.

How to read it in a meeting

Left to right is the **data path**. Bottom to top is the **control path**. Almost every question is one of four:

- "Where does X land?" → a column
- "Who can see it?" → Lake Formation
- "How do I know it ran?" → orchestration and observability
- "What happens when it breaks?" → isolation (D24 in Module 0) plus idempotency

Being able to point at the right box is most of the job.

D01 · The Lakehouse Reference Architecture

Checklist

- ☐ I can draw the five columns from memory
- ☐ I can name three services in each
- ☐ I can name the five cross-cutting layers
- ☐ I know which layer is loaded into Redshift and which is pointed at

- ☐ I can answer the four common questions by pointing

You've got it when you can...

...draw this on a whiteboard in five minutes and answer "where would a new source go?" by pointing rather than thinking.

Four Paradigms, Four Architectures

The same four words from Module 0 — but drawn as the services each one actually resolves to on AWS.

1 · SAME QUESTION, FOUR ANSWERS

WAREHOUSE



Source database
OLTP, row-oriented



ETL
model first, load second



Amazon Redshift
schema on write



BI
numbers that reconcile

USE WHEN

the shape is known and the numbers must reconcile

BUY: TRUST

LAKE



Any files
any shape, as received



Amazon S3
cheap, immutable



Glue Data Catalog
schema on read



Athena
priced per TB scanned

USE WHEN

you must keep everything and decide meaning later

BUY: OPTIONALITY

LAKEHOUSE



S3 Tables
managed Iceberg



Glue / EMR
MERGE, never append



Redshift
gold layer only



Many engines
one copy of the table

USE WHEN

you need lake economics and warehouse guarantees

BUY: BOTH — THIS IS US

MESH



Lake Formation
cross-account grants



AWS RAM
shares the resource



Datashares
live, across accounts



Domain teams
own their data products

USE WHEN

there are many domain teams — not one platform team

BUY: AUTONOMY

IN PLAIN ENGLISH

Three of these are places to put data. The fourth is a rule about who owns it. Only the third one is a building we are actually putting up.

TeachMe-Lakehouse · Module 3 · Architecture 02

Foundation Architectures

D02 · Four Paradigms, Four Architectures

Diagram: [_render/D02-four-paradigms.html](#)

What it shows

The four words from Module 0 — warehouse, lake, lakehouse, mesh — drawn as the **services each one actually resolves to on AWS**. Definitions are easy to nod along to; four stacks side by side are harder to be vague about.

The four stacks

	WAREHOUSE	LAKE	LAKEHOUSE	MESH
Source	RDS / OLTP	any files	S3 Tables	—
Middle	ETL, model first	S3, cheap and immutable	Glue / EMR, MERGE not append	Lake Formation + RAM
Store	Amazon Redshift	Glue Data Catalog	Redshift, gold only	Redshift datashares
Read	BI, trusted numbers	Athena, per TB scanned	many engines, one table	domain teams
You buy	trust	optionality	both	autonomy

The one that matters

Lakehouse. Lake economics with warehouse guarantees, bridged by an open table format. It is what Tamimi is and what Apparel Group will be.

The reason is not fashion. With eight unfamiliar source systems you will be wrong about the meaning of fields, repeatedly. A pure warehouse makes every correction a migration; a pure lake cannot give finance a number they will sign. A lakehouse lets you land cheaply and model the part that needs modelling, without moving the data to do it.

Mesh is drawn dashed on purpose

Because it is not a place to put data. It is an ownership model that sits **on top of** one of the other three. The AWS pieces are real — Lake Formation grants over RAM, datashares, federated catalogs (D27) — but they implement the sharing, not the organisation.

Learn it so that when it is proposed you can ask the useful question: *which domain teams would own which products, and who is on call for them?*

The trade to say out loud

Every option has a characteristic bad day:

- **Warehouse** — a schema migration nobody scoped.
- **Lake** — discovering the file format changed upstream two months ago.
- **Lakehouse** — compaction, snapshot expiry, and a catalog that must be kept correct.
- **Mesh** — a domain team that stopped answering.

Choose the bad day you can staff.

Checklist

- ☐ I can name what each option *buys* in one word
- ☐ I can name the AWS services in each stack
- ☐ I can explain why a lakehouse suits eight unfamiliar sources
- ☐ I know why mesh is drawn dashed
- ☐ I can name each option's characteristic bad day

D02 · Four Paradigms, Four Architectures

You've got it when you can...

...sit in a design review where four people advocate four architectures and reframe it from "which is best" to "which trade-off are we willing to operate".

The Medallion Layers

Four layers, and the promise each one makes. What matters is not the names — it is what changes at every boundary.

1 · WHAT EACH LAYER PROMISES



WHAT CHANGES AT EACH BOUNDARY — THIS IS THE ACTUAL LESSON

raw → bronze: **types and a key** · bronze → silver: **meaning and joins** · silver → gold: **a model, and a promise about the number**

IN PLAIN ENGLISH

Delivered, unpacked, sorted, priced. You can walk back to any earlier stage — which is exactly why nothing is ever thrown away.

D03 · The Medallion Layers

Diagram: [_render/D03-medallion-layers.html](#)

What it shows

Four layers and **the promise each one makes**. The names are not the lesson — plenty of teams have raw/bronze/silver/gold prefixes and no idea what distinguishes them. The lesson is what changes at every boundary.

The four promises

RAW — *"we can always rebuild everything from here"*

S3, plain files, exactly as received. Immutable, written once, no schema enforced, kept for years. **Never queried by a report**. This is evidence, not data.

BRONZE — *"re-running a day changes nothing"*

Iceberg tables. Types applied, columns named, MERGE on the natural key, one row per source record. Still source-shaped — no business meaning has been added yet.

SILVER — *"a column means the same thing in every table"*

Iceberg tables, conformed and joined. Business rules applied, sources joined into entities, one meaning per column. Read in place by Spectrum rather than copied into the warehouse.

GOLD — *"the same question gives the same answer twice"*

Redshift tables. Dimensionally modelled, Type 2 history preserved, loaded rather than pointed at. The only layer BI reads.

What changes at each boundary

raw → bronze: types and a key bronze → silver: meaning and joins silver → gold: a model, and a promise about the number

If you cannot say which of those three a piece of logic belongs to, it is in the wrong layer. That single test resolves most "where should this transformation live?" arguments.

Why raw is never deleted

Because bronze, silver and gold are all **derived**. When a business rule turns out to have been wrong for six months, you fix the rule and reprocess — and nothing was lost, because nothing downstream was ever the only copy (D30).

That property is the entire justification for paying to keep raw. It is not hoarding; it is the second undo button.

The common failure

Skipping silver. Teams go bronze → gold directly, and the conforming logic ends up duplicated inside three different gold models, drifting apart. The symptom is two dashboards disagreeing about "region", and the cause is that region was defined three times.

Checklist

- ☐ I can state each layer's promise in one sentence
- ☐ I know what changes at each of the three boundaries
- ☐ I can place a given transformation in the right layer
- ☐ I know why raw is immutable and retained
- ☐ I would notice bronze → gold with no silver

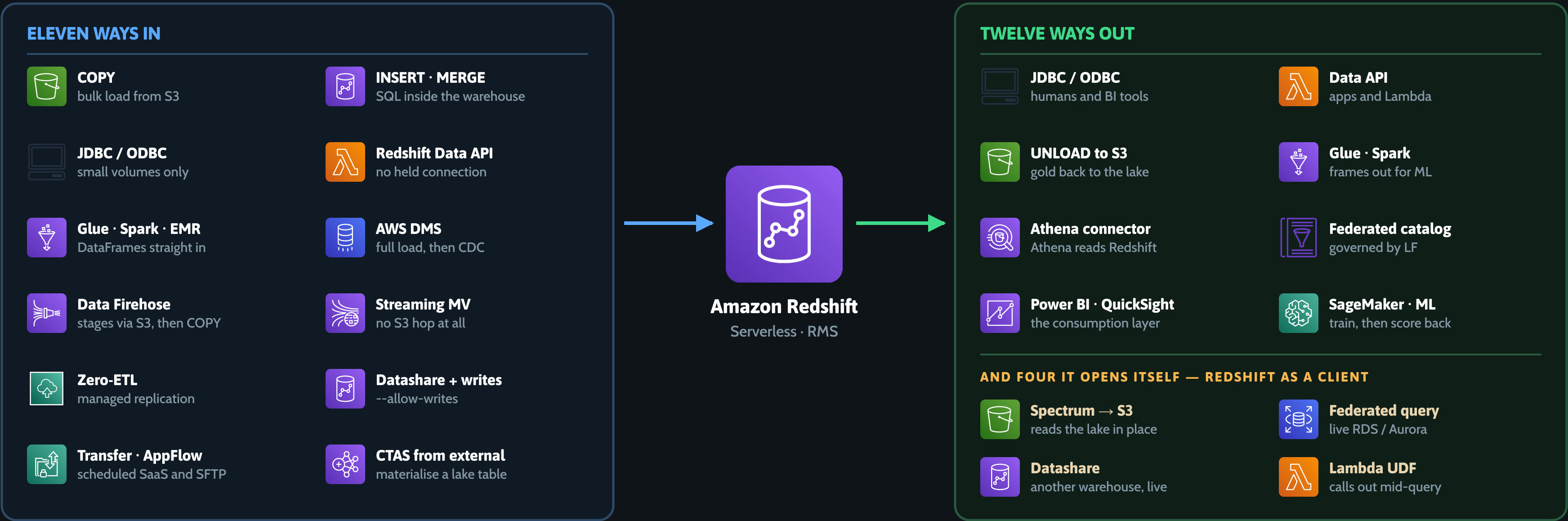
D03 · The Medallion Layers

You've got it when you can...

...be shown a piece of business logic and say which boundary it belongs to — and explain what breaks if it is put one layer too early or too late.

Who Writes, Who Reads

Every path into and out of the warehouse, on one canvas. Eleven ways in, twelve ways out — and four of those it opens itself.



IN PLAIN ENGLISH

Eleven loading bays, twelve exits, and four windows it can see out of. If you are writing custom code to move data, check this picture first.

D04 · Who Writes, Who Reads

Diagram: [_render/D04-who-reads-who-writes.html](#)

What it shows

Every path into and out of Redshift on one canvas: **eleven ways in, twelve ways out — and four the warehouse opens itself.**

Most people know three or four of these and assume the rest do not exist. That is how you end up with a Lambda holding a JDBC connection, or a weekly CSV export that somebody maintains by hand.

Eleven ways in

`COPY` · `INSERT / MERGE` · JDBC/ODBC · Redshift Data API · Glue/Spark/EMR connector · AWS DMS · Data Firehose · streaming ingestion · zero-ETL · datashare with `--allow-writes` · Transfer Family and AppFlow

Two worth singling out: - **Data API** — HTTP, no held connection, no driver. The right choice from Lambda and Step Functions, and the one people reach for JDBC instead of. - **Streaming ingestion** — a materialized view mapped straight onto Kinesis or MSK, with **no S3 hop at all.**

Twelve ways out

JDBC/ODBC · Data API · `UNLOAD` · Glue/Spark · Athena connector · Glue federated catalog · Power BI/QuickSight · SageMaker and Redshift ML — plus the four below.

`UNLOAD` is the most overlooked thing on this diagram. Publishing gold back to S3 as Parquet is what stops Redshift becoming the only door to the data. Every unofficial spreadsheet in an organisation started life as a missing exit.

Four it opens itself

The direction people forget — Redshift is not only a destination:

	REACHES
Spectrum	S3 external tables, through the catalog
Federated query	live RDS / Aurora PostgreSQL or MySQL
Datashare	another warehouse, live
Lambda UDF	a function, mid-query

These are what make "get the data into Redshift" a *choice* rather than a prerequisite (D07).

Three consequences

1. **You rarely need to invent a path.** If you are writing custom code to move data in or out, check this picture first.
2. **The security surface is bigger than the database password.** Eleven ways in means eleven things to govern (D19, D28).
3. **Zero-copy changes architecture.** Spectrum, federated query and datashares mean loading is a decision with a cost, not a default.

Checklist

- ☐ I can name at least eight write paths
- ☐ I can name at least six read paths
- ☐ I know Redshift can read S3, live databases and other warehouses
- ☐ I know `UNLOAD` exists and what it prevents
- ☐ I use the Data API from Lambda, not JDBC
- ☐ I have this diagram somewhere I will see it

D04 · Who Writes, Who Reads

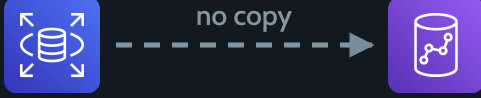
You've got it when you can...

...be asked "how do we get X into Redshift?" and answer with the right mechanism and the reason — including the cases where the answer is "we don't; we point at it".

Six Ways To Move Data

Every mechanism, source to target, with the latency it buys. Latency improves down the list; the control you keep gets worse.

1 · SOURCE → MECHANISM → TARGET

MECHANISM	THE PATH	LATENCY	CHOOSE IT WHEN
BATCH ETL		hours	You must transform on the way and control the shape — the medallion default
CDC REPLICATION		minutes	Continuous replication that catches hard deletes, reading the log not the tables
ZERO-ETL		min - 1 hr	You want the table there with no pipeline at all — and will shape it after landing
FEDERATED QUERY		live	A small lookup against a live database — filters push down, large scans do not
STREAMING		seconds	Genuinely event-driven data where seconds change a decision — rare in retail
FILE TRANSFER		scheduled	SFTP drops, NAS sync and SaaS connectors you would rather not write yourself

IN PLAIN ENGLISH

Pick the latency you actually need, then take the most managed option that still gives you it. Anything faster is a bill with no reader.

D05 · Six Ways To Move Data

Diagram: [_render/D05-six-ways-to-move-data.html](#)

What it shows

Every mechanism drawn as **source** → **mechanism** → **target**, with the latency each one buys.

Latency improves down the list. The control you keep gets worse. Those two move together, and no configuration separates them.

The six

MECHANISM	SERVICE	LATENCY	CHOOSE IT WHEN
Batch ETL	Glue · EMR	hours	you must transform on the way — the medallion default
CDC replication	AWS DMS	minutes	continuous, and the source hard-deletes
Zero-ETL	managed integration	min – 1 hr	you want it there with no pipeline at all
Federated query	Redshift · Athena	live	a small lookup against a live database
Streaming	Kinesis · MSK · Firehose	seconds	genuinely event-driven — rare in retail
File transfer	Transfer Family · DataSync · AppFlow	scheduled	SFTP drops and SaaS you would rather not code

The rule

Pick the latency you actually need, then take the most managed option that still gives you it.

Note the shape of the federated-query row on the diagram: a **dashed arrow with no middle icon**. Nothing is moved. That is the visual reminder that it is a pointer, and that the cost lands on the source system every time somebody runs the query.

How to use it in a design review

Four questions, in order:

1. **What latency does the *decision* need?** Not what was requested. "Real-time" in a requirements document usually means "not yesterday's". Find out what someone does differently at 9am than at 9pm.
2. **Does the shape need to change on the way?** If yes, managed replication is out for that source.
3. **Who operates it at 3am?** A path with no code has no code to debug, and that has real value.
4. **What happens when volume triples?** Per-row costs and always-on costs scale very differently.

The comparison that matters for us

Rows one, two and three against the **same Oracle source** — that is D17, and it is a genuine decision with arguments on both sides rather than a preference.

D05 · Six Ways To Move Data

Checklist

- ☐ I can reproduce the six-row table from memory
- ☐ I can state the latency-versus-control rule in one sentence
- ☐ I know which mechanism reads the source's log rather than its tables
- ☐ I know zero-ETL supports Oracle and SAP, not just Aurora

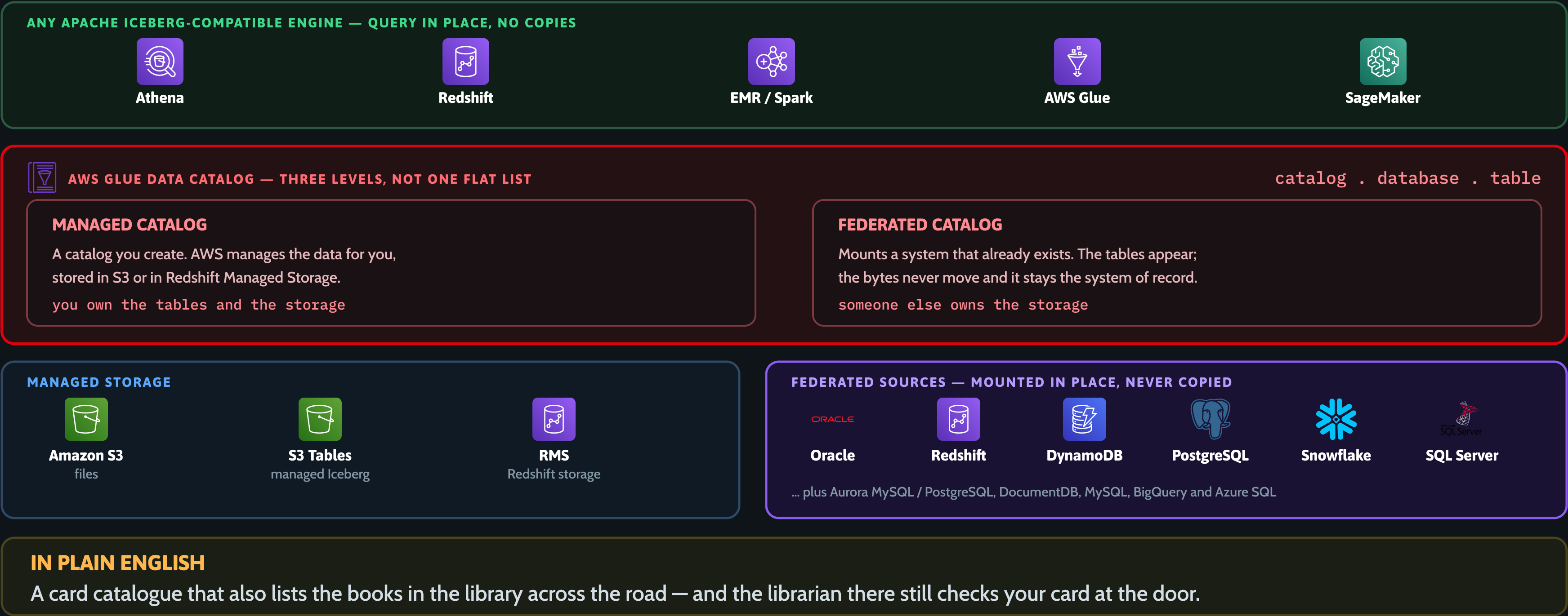
- ☐ I ask what latency the decision needs, not what was requested
- ☐ I consider who operates it at 3am

You've got it when you can...

...enter a design review where a mechanism has already been chosen, ask the four questions, and either confirm it with reasons or change it — inside ten minutes.

The Federated Catalog

Engines on top, storage underneath, and one governed catalog in between — which can now mount systems it does not own.



D06 · The Federated Catalog

Diagram: [_render/D06-federated-catalog.html](#)

What it shows

Three layers: **engines on top, storage underneath, and one governed catalog in between** — which can now mount systems it does not own.

If your mental model of the Glue Data Catalog is "a place that remembers table schemas so Athena can query S3", it is several years out of date.

Three levels, not one flat list

```
catalog . database . table
```

With **nested catalogs** allowed, so the hierarchy can mirror the shape of the system underneath.

Two kinds of catalog

Managed catalog — one you create. AWS manages the data, stored in **S3** or **Redshift Managed Storage**. You own the tables and the storage.

Federated catalog — mounts a system that already exists. The tables appear and become queryable; **the bytes never move** and it remains the system of record. Someone else owns the storage.

What you can mount

Amazon Redshift · DynamoDB · DocumentDB · MySQL · PostgreSQL · SQL Server · **Oracle** · Aurora MySQL · Aurora PostgreSQL · Google BigQuery · Snowflake · Microsoft Azure SQL — plus **S3 Tables** as `s3tablescatalog`.

Oracle on that list matters for us. It means Apparel Group's Oracle reference and lookup data could be *mounted* rather than ingested — a third option alongside a Glue pipeline and zero-ETL (D17), worth considering per table rather than per source.

One governance plane

Lake Formation grants at **catalog** → **database** → **table** → **column**, and the grant is **enforced by every engine that reads through the catalog** — Athena, Spark, EMR, Redshift Spectrum. Define once, applies everywhere.

Compare the alternative: the same rule re-implemented in four places, drifting apart, with nobody able to say which one is authoritative.

Any Iceberg-compatible engine

Because both the catalog and the table format are open, you query from whichever engine suits the job against **one copy** of the data. Athena to explore, Redshift to report, Spark to transform — no exports, no drift.

This is the **SageMaker Lakehouse architecture**: S3 data lakes and Redshift warehouses unified into one catalog.

 Naming in this area has moved recently (SageMaker Lakehouse / Unified Studio / DataZone). Check current AWS documentation before quoting product names in a client deck.

D06 · The Federated Catalog

Two gotchas

- **Lowercase identifiers.** The lakehouse architecture currently supports lowercase table, column and database names. Plan naming conventions accordingly — retrofitting is painful.
- **Prefer declarative registration to crawlers.** A crawler infers schema by sampling, so your table definition can change because the *data* changed, silently. Register explicitly and a schema change becomes a pull request.

Checklist

- ☐ I can state the three levels
- ☐ I know the difference between managed and federated catalogs

- ☐ I can name at least six mountable sources
- ☐ I know Lake Formation grants are enforced across engines
- ☐ I know about the lowercase constraint
- ☐ I know why mature platforms avoid crawlers

You've got it when you can...

...be asked "how do we get the Oracle reference tables into the lake?" and offer three real options — ingest, replicate, or mount — with the trade-offs of each.

Zero-Copy Access Paths

Four ways to query data you never loaded. None of them is free — each one just moves the bill somewhere else.

1 · FOUR DOORS, FOUR DIFFERENT PAYERS

SPECTRUM



S3 / Iceberg

Redshift

External schema over the Glue catalog. Reads Parquet, ORC, CSV, JSON, Avro — plus Iceberg, Hudi CoW and Delta.

YOU PAY FOR

bytes scanned in S3, on every single query

BEST FOR

cold history you scan rarely

FEDERATED QUERY



RDS / Aurora

Redshift

External schema onto a live PostgreSQL or MySQL database. Predicates are pushed down to the source. Large scans are not.

YOU PAY FOR

load on the production database, every query

BEST FOR

small, live lookups only

DATASHARE



Producer

Consumer

Live data across clusters, accounts and Regions. No copy, no ETL, no lag — and writes are now possible with allow-writes.

YOU PAY FOR

the consumer's own compute, not yours

BEST FOR

another team's warehouse

ATHENA CONNECTORS



Anything

Athena

Redshift, DynamoDB, Snowflake and more. Two connector types — only the Glue-connection kind supports Lake Formation.

YOU PAY FOR

Athena scan cost, plus the remote system

BEST FOR

one SQL door to everything

IN PLAIN ENGLISH

A library card, not a photocopy. You can read the book any time — but you walk to the other building every single time you open it.

D07 · Zero-Copy Access Paths

Diagram: [_render/D07-zero-copy-paths.html](#)

What it shows

Four ways to query data you never loaded — and, for each, **who actually pays**.

None of them is free. Each one moves the bill somewhere else.

"Zero-copy" is a statement about storage, not about cost. Teams who hear it as "zero-cost" build designs that demo beautifully and degrade quietly.

The four doors

Spectrum → S3

External schema over the Glue catalog. Reads Parquet, ORC, CSV, JSON, Avro — plus the transactional formats **Iceberg**, **Hudi Copy-on-Write** and **Delta Lake** (via symlink manifests). **You pay:** bytes scanned in S3, on every query. **Best for:** cold history you scan rarely.

Federated query → a live database

External schema onto a live **RDS** or **Aurora PostgreSQL / MySQL**. Predicates are pushed down to the source; large scans are not. **You pay:** load on the production database, every query. **Best for:** small, live lookups only.

Datashare → another warehouse

Live data across clusters, accounts and Regions. No copy, no ETL, no lag — and **writes are now possible** with `--allow-writes`. **You pay:** the consumer's own compute, not yours. **Best for:** another team's warehouse.

Athena connectors → anything

Redshift, DynamoDB, Snowflake and more. **Two connector types** — only the Glue-connection kind registers as a federated catalog and supports Lake Formation fine-grained control. Prefer that one. **You pay:** Athena scan cost plus the remote system. **Best for:** one SQL door to everything.

The decision framework

Four questions, in order:

1. **How often is it queried?** Many times a day → lean toward loading. Twice a month → point at it.
2. **Is it joined, or just filtered?** Heavy joins want co-located data and sort keys.
3. **How fresh must it be?** "Right now" narrows you to federated query or a datashare.
4. **Who pays, and do they know?** A pointer moves cost onto the querier — or onto someone else's production system.

Copy what you join hard and often. Point at what you scan rarely. Never point at production for a report that runs every hour.

The failure mode

Someone points at a source for a small lookup. The report gets popular. It goes from weekly to hourly. Six months later a production database is struggling and nobody connects it to the dashboard.

The defence: write down, per external schema, what it is for and roughly how often it should be hit — then alarm when reality diverges.

D07 · Zero-Copy Access Paths

Checklist

- ☐ I can name the four doors and what each reaches
- ☐ I can say who pays for each
- ☐ I know Spectrum reads Iceberg, Hudi CoW and Delta
- ☐ I know the two Athena connector types differ on governance

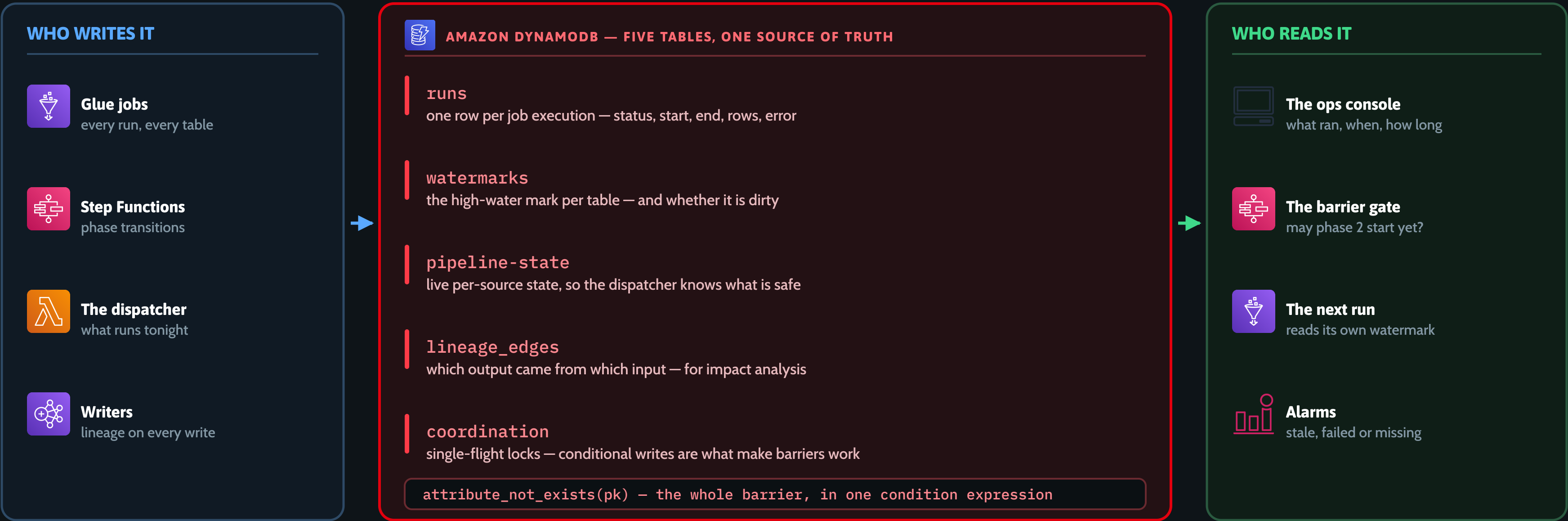
- ☐ I decide per table, not once for the platform
- ☐ Each external schema has a documented purpose

You've got it when you can...

...be asked to "just point Redshift at production" for a daily report, explain what that does to production, and offer the right alternative instead.

The Control Plane

Five DynamoDB tables that turn “did last night work?” into a query instead of a log search — and make barriers possible at all.



IN PLAIN ENGLISH

The clipboard by the door. Anyone can read what happened last night without going through the bins — and two people cannot take the same job.

D08 · The Control Plane

Diagram: [_render/D08-control-plane.html](#)

What it shows

Five DynamoDB tables, who writes them, and who reads them. Together they turn "did last night work?" from a log search into a query — and they are what makes barriers possible at all.

The five tables

TABLE	HOLDS
runs	one row per job execution — status, start, end, rows, error
watermarks	the high-water mark per table, and whether it is dirty
pipeline-state	live per-source state, so the dispatcher knows what is safe
lineage_edges	which output came from which input, for impact analysis
coordination	single-flight locks — conditional writes are what make barriers work

Who writes

Glue jobs (every run, every table) · Step Functions (phase transitions) · the dispatcher (what runs tonight) · writers (lineage on every write).

Who reads

The ops console (what ran, when, how long) · the barrier gate (may phase 2 start?) · the next run (reads its own watermark) · alarms (stale, failed or missing).

The one line that matters

```
ConditionExpression: attribute_not_exists(pk)
```

The first writer wins; a second attempt fails harmlessly. That single expression gives you single-flight locks, idempotent phase transitions, and safety against two schedulers firing at once (D25).

Why this is not "just logging"

Logs are for reading **after** you already know something is wrong. The control plane is for **asking**:

- Did XStore load last night? → a query
- What is the watermark for this table? → a query
- Is it safe to start phase 2? → a query
- If I change this source, what breaks? → a query against lineage

Every one of those becomes a person's time if the answer only exists in CloudWatch Logs.

The dirty-watermark guard

`watermarks` carries more than a value — it carries whether the last run left it **dirty**. A job that failed part-way must not let the next run advance past data it never wrote. That flag is the difference between a failed load and a silent gap.

D08 · The Control Plane

Checklist

- ☐ I can name the five tables and what each holds
- ☐ I know who writes each and who reads each
- ☐ I can explain the conditional write and what it prevents
- ☐ I know why a watermark carries a dirty flag

- ☐ I can answer "did it run?" with a query, not a log search
- ☐ Lineage is written on every load, not reconstructed later

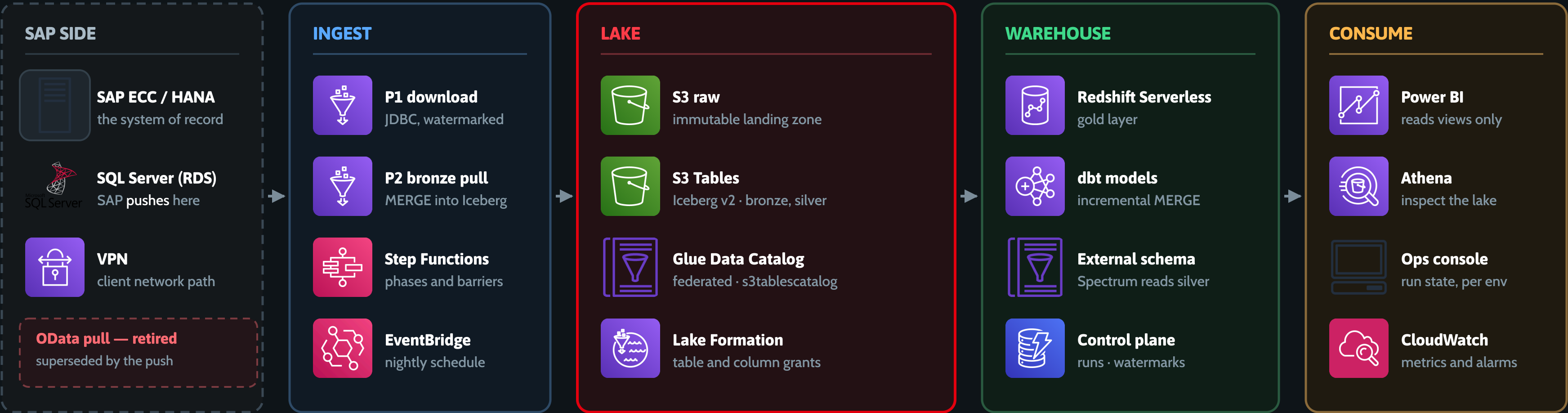
You've got it when you can...

...be asked at 7am whether last night worked, and answer in ten seconds from a query — then say which table you looked at and why.

Tamimi, End To End

The platform that exists today. SAP pushes to a staging database; everything after that is ours.

1 · THE DATA PATH IN PRODUCTION



THE THING TO NOTICE

SAP pushes to SQL Server — we never poll SAP directly. Everything from that database rightwards is ours to design, operate and change.

IN PLAIN ENGLISH

The supplier leaves the delivery at an agreed door. We never walk into their warehouse — which is why their upgrades do not become our outages.

D09 · Tamimi, End To End

Diagram: [_render/D09-tamimi-end-to-end.html](#)

What it shows

The platform **that exists today** — not a target state. This is the handover record: if the team changes, this diagram is what a new joiner reads first.

The SAP boundary

The most important thing on this diagram is on the left.

SAP pushes to a SQL Server database on RDS. We never poll SAP directly.

That is a deliberate boundary, and it is why SAP's upgrade windows do not become our outages. Everything from that staging database rightwards is ours to design, operate and change; everything to the left of it is theirs.

The earlier **OData pull is retired** — drawn dashed on the diagram so nobody re-proposes it in a design session six months from now. It is history, and marking history as history on the diagram is cheaper than explaining it repeatedly.

The rest of the path

Ingest — Glue P1 (`source_download` , JDBC, watermarked) and P2 (`bronze_pull` , MERGE into Iceberg), sequenced by Step Functions with a barrier between them (D10), triggered nightly by EventBridge.

Lake — S3 raw as an immutable landing zone; S3 Tables holding Iceberg v2 bronze and silver; the federated Glue catalog (`s3tablescatalog`); Lake Formation grants.

Warehouse — Redshift Serverless for gold, built by dbt with incremental MERGE. An external schema lets Spectrum read silver in place rather than loading it. The control plane records what happened.

Consume — Power BI reads views only; Athena for investigation; the ops console for run state per environment; CloudWatch for metrics and alarms.

How to use this diagram

Three situations:

1. **Onboarding.** New joiner, day one. Walk left to right, then point at the band underneath.
2. **Incident.** "Where did it break?" — point at the column, then open D10 for the phase detail.
3. **Change request.** "Can we add X?" — find which column it lands in, then check D12 for the catalog implications.

What it deliberately does not show

Failure paths, retries and barriers — those are D10. Environments — that is D13. This diagram answers *what is connected to what*, and nothing else. A diagram that tries to show everything shows nothing.

Checklist

- ☐ I can explain the SAP push boundary and why it exists
- ☐ I know the OData pull is retired and would not re-propose it
- ☐ I can name what P1 and P2 each do
- ☐ I know which layers are Iceberg and which is Redshift
- ☐ I know Power BI reads views, not tables
- ☐ I can walk this diagram for a new joiner without notes

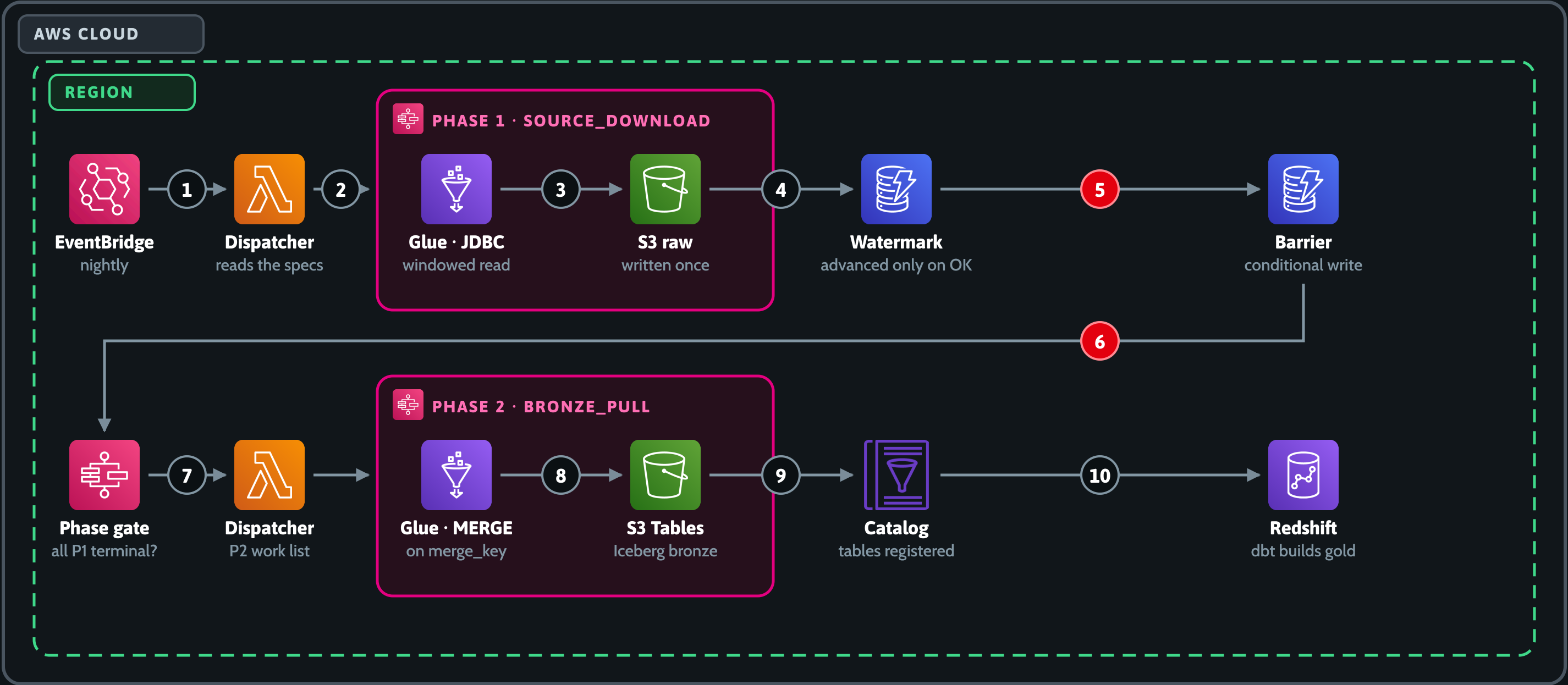
D09 · Tamimi, End To End

You've got it when you can...

...hand this to someone on their first morning, walk it in ten minutes, and have them able to ask a sensible question about where their work will land.

The P1 / P2 Split

Two phases, one barrier between them. Ten steps — and step six is the only reason the whole thing is safe to re-run.



IN PLAIN ENGLISH

Unload every lorry before anyone starts stacking shelves. The barrier is the foreman saying “all in” — and only one foreman can say it.

D10 · The P1 / P2 Split

Diagram: [_render/D10-p1-p2-split.html](#)

What this pattern is for

Two phases with a barrier between them. **Every source must finish landing before any source starts building** — because a bronze table built while its siblings are still arriving is a table built on a partial view.

The ten steps

- 1 · **EventBridge fires, once.** One schedule, one entry point. Not one cron per source — that is how you get two jobs racing at 02:00 and a schedule nobody can reason about.
- 2 · **The dispatcher reads the specs.** It works out which sources are due from configuration and control-plane state, not from a hardcoded list. This is what makes adding a source a config change.
- 3 · **Glue reads over JDBC, windowed.** Each source in its own branch, reading a bounded window rather than the whole table. Parallelism is sized per source (Module 2 LO3), because RMS and XStore do not deserve the same settings.
- 4 · **Raw is written once.** Immutable, exactly as received. Nothing downstream is the only copy from this moment on.
- 5 · **The watermark advances — only on success.** ★ This is the correctness hinge of phase 1. A job that failed part-way must **not** advance its watermark, or the next run skips data that was never written. The watermark carries a **dirty** flag for exactly this case.

6 · **The barrier.** ★ Each branch records its terminal state with a **conditional write** — `attribute_not_exists(pk)` . First writer wins; a duplicate attempt fails harmlessly. This is the entire single-flight mechanism, and it is why two dispatchers cannot both open the gate.

7 · **The phase gate asks one question.** Are all phase-1 branches **terminal**? Not *successful* — terminal, which includes failed. A failed source must be visible and skipped, never able to hold the gate closed forever. Otherwise one broken feed stalls the whole platform.

8 · **Phase 2 MERGEs into Iceberg.** On `merge_key` . Because it merges rather than appends, re-running a branch is safe — which is what turns an incident into a retry rather than a repair.

9 · **Tables are registered in the catalog.** Declaratively. The schema is a reviewed artefact, not something a crawler inferred from whatever happened to land.

10 · **dbt builds gold in Redshift.** Only after phase 2 completes. Gold is the layer that promises the same answer twice, and it cannot promise that on top of a half-built silver.

Why two phases and not one

Because **landing and building have different failure modes and different retry costs.**

Landing is network-bound and fails for reasons outside your control — a source is slow, a VPN drops, credentials expire. Building is compute-bound and fails for reasons inside it — a bad merge key, a type mismatch.

Separating them means a slow source delays the build; it does not corrupt it. Merging them means every network hiccup becomes a partially-built table.

D10 · The P1 / P2 Split

What breaks if you skip a piece

- **No barrier** — bronze is built on a partial landing, silently.
- **Gate on success rather than terminal** — one broken source blocks everything, indefinitely.
- **Watermark advances on failure** — a permanent gap nothing will ever tell you about.
- **Append instead of MERGE** — a retry doubles the day.

Checklist

- ☐ One schedule, one entry point

- ☐ Watermarks advance only on success, and carry a dirty flag
- ☐ The barrier is a conditional write
- ☐ The gate waits for **terminal**, not for success
- ☐ Phase 2 MERGEs on a key and is safe to re-run
- ☐ I have deliberately failed one source and watched the gate behave

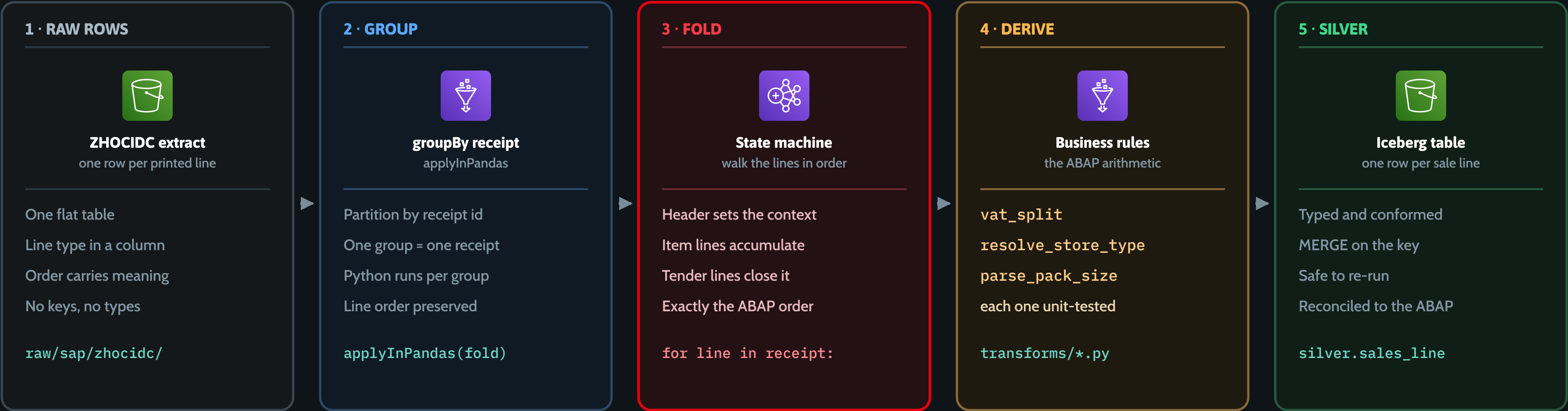
You've got it when you can...

...explain to a new engineer why phase 2 does not simply start when phase 1's *first* branch finishes — and point at the row in DynamoDB that stops it.

ABAP To Glue, Stage By Stage

Twenty years of receipt logic, moved to Spark without changing a single number. The order of the lines is the business rule.

1 · FIVE STAGES, ONE INVARIANT



THE ONLY ACCEPTANCE TEST THAT MATTERS

Feed a day of real receipts through both the ABAP report and the Glue job. **Row for row, the totals must match** — anything else is a rewrite, not a migration.

IN PLAIN ENGLISH

A till roll only makes sense read top to bottom. Shuffle the lines and the receipt stops meaning anything — which is why we fold, not aggregate.

D11 · ABAP To Glue, Stage By Stage

Diagram: [_render/D11-abap-to-glue.html](#)

What it shows

Twenty years of receipt logic moved to Spark **without changing a single number**. Five stages, and one invariant that has to hold throughout.

The invariant

The order of the lines *is* the business rule.

A till roll only makes sense read top to bottom. A header sets the context; item lines accumulate against it; tender lines close it. Shuffle the rows and the receipt stops meaning anything.

That is why this is a **fold**, not an aggregation. Most Spark instincts — `groupBy().agg()` — throw away exactly the thing that carries the meaning.

The five stages

1 · Raw rows. The ZHOCIDC extract: one flat table, one row per printed line, line type in a column, no keys and no types. Order carries meaning.

2 · Group. `groupBy(receipt_id).applyInPandas(...)` . Partition by receipt so one group is one complete receipt, and Python runs per group with line order preserved inside it.

3 · Fold. Walk the lines in order, exactly as the ABAP did. Header sets context, items accumulate, tenders close. This is a positional state machine, and it is the part that must match the original statement for statement.

4 · Derive. The ABAP arithmetic, extracted into named, unit-tested functions: `vat_split` , `resolve_store_type` , `parse_pack_size` . Each one small enough to test in isolation, which is what makes the whole thing reviewable.

5 · Silver. An Iceberg table, one row per sale line, typed and conformed, MERGE*d* on the key so it is safe to re-run.

The only acceptance test that matters

Feed a day of real receipts through **both** the ABAP report and the Glue job.

Row for row, the totals must match.

Anything else is a rewrite, not a migration — and a rewrite is a much bigger conversation with the business than anyone signed up for.

Where they legitimately differ (a rounding rule you deliberately corrected), **write the divergence down and pin it with a unit test**, so the next person does not "fix" it back.

Why the derived functions are separate

Three reasons, all practical:

- **Testable.** `vat_split` takes numbers and returns numbers. No Spark, no fixtures, milliseconds per test.
- **Reviewable.** A finance person can read a twelve-line function. They cannot read a Spark job.
- **Reusable.** The same rule applies in more than one place, and now it exists once.

D11 · ABAP To Glue, Stage By Stage

Checklist

- ☐ I can explain why this is a fold rather than an aggregation
- ☐ I know why `applyInPandas` preserves what `groupBy().agg()` destroys
- ☐ Business rules are extracted into named, unit-tested functions
- ☐ The reconciliation test compares against the ABAP, row for row

- ☐ Deliberate divergences are documented and pinned by a test
- ☐ The silver table MERGEs and is safe to re-run

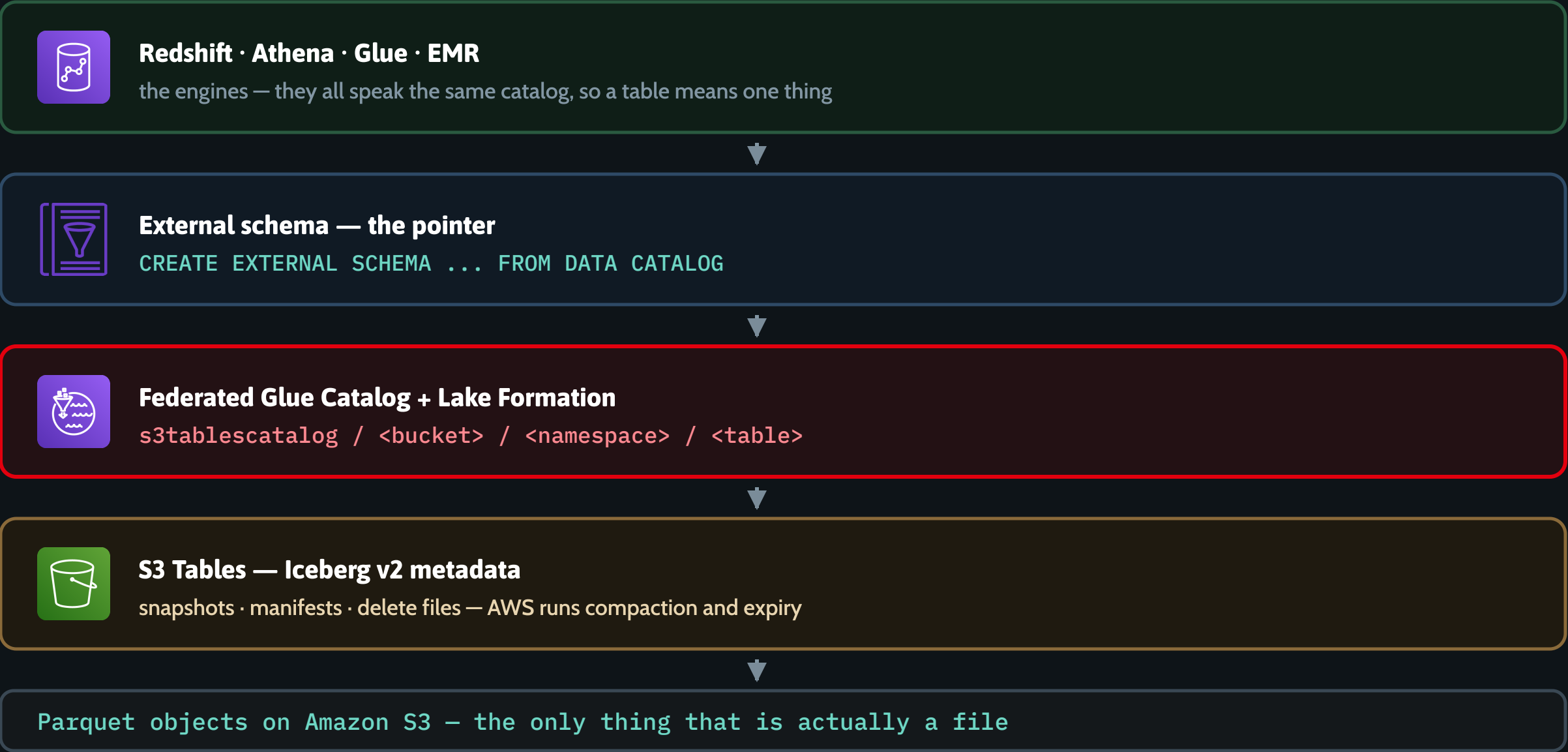
You've got it when you can...

...be handed another ABAP report and immediately ask the right first question: *does the order of the rows carry meaning?* — because the answer decides the entire shape of the job.

Catalog And Storage Layers

Five layers between a SQL statement and a byte on disk. Knowing which layer you are debugging is most of the job.

1 · TOP TO BOTTOM



WHERE IT BITES

The Spectrum mirror database

Spectrum resolves the catalog with the account baked in, so a federated catalog cannot be referenced directly. A mirror database is created for Redshift to point at instead.

Two grant systems, one table

Redshift GRANT governs its own tables. Lake Formation governs anything read through the catalog. “Works in Athena, not in Redshift” starts here.

Maintenance is not optional

Iceberg v2 writes delete files rather than rewriting data. Without compaction and snapshot expiry, reads get slower and storage grows forever.

IN PLAIN ENGLISH

Address, street index, town plan, land registry, and the actual bricks. When something is missing, first work out which of the five you are looking at.

D12 · Catalog And Storage Layers

Diagram: [_render/D12-catalog-storage-layers.html](#)

What it shows

Five layers between a SQL statement and a byte on disk. When something does not work, knowing which of the five you are debugging is most of the job.

LAYER	WHAT IT IS
Engines	Redshift · Athena · Glue · EMR — they all speak the same catalog
External schema	the pointer — <code>CREATE EXTERNAL SCHEMA ... FROM DATA CATALOG</code>
Federated catalog	<code>s3tablescatalog / <bucket> / <namespace> / <table></code> + Lake Formation
Iceberg metadata	snapshots, manifests, delete files — S3 Tables runs compaction and expiry
S3 objects	Parquet files — the only layer that is actually a file

Three places it bites

The Spectrum mirror database

Spectrum resolves the catalog with the account baked in, so a federated catalog cannot be referenced directly. A **mirror database** is created for Redshift to point at instead.

This is not a workaround somebody invented; it is a documented consequence of how Spectrum resolves catalog IDs. It is also the single most confusing thing in the stack for a newcomer, because the table exists, the grant exists, and Redshift still cannot see it.

Two grant systems, one table

Redshift `GRANT` governs Redshift-native tables. **Lake Formation** governs anything read through the catalog. *"Works in Athena, not in Redshift"* almost always starts here (Module O L17, D28).

Maintenance is not optional

Iceberg v2 writes **delete files** rather than rewriting data files. Without compaction and snapshot expiry, reads get slower and storage grows for data that is logically gone. S3 Tables handles this — which is a large part of why it is worth using — but you still need to know it is happening.

How to debug with this diagram

Work **down**, not sideways:

1. Can the engine see the schema? → external schema layer
2. Can it see the table? → catalog layer
3. Is it allowed to? → Lake Formation
4. Is the data actually there? → Iceberg metadata (check the snapshot)
5. Are the files present? → S3

Four of those five questions are answered without opening S3 at all. People who skip to step 5 spend a long time looking at objects that were never the problem.

Supersedes

This diagram replaces the hand-drawn [../..//diagrams/catalog-storage-layers.drawio](#) — same content, real AWS icons, consistent with the rest of the pack.

D12 · Catalog And Storage Layers

Checklist

- ☐ I can name the five layers in order
- ☐ I know why a Spectrum mirror database exists
- ☐ I know which grant system governs which kind of object
- ☐ I know Iceberg v2 uses delete files and why compaction matters

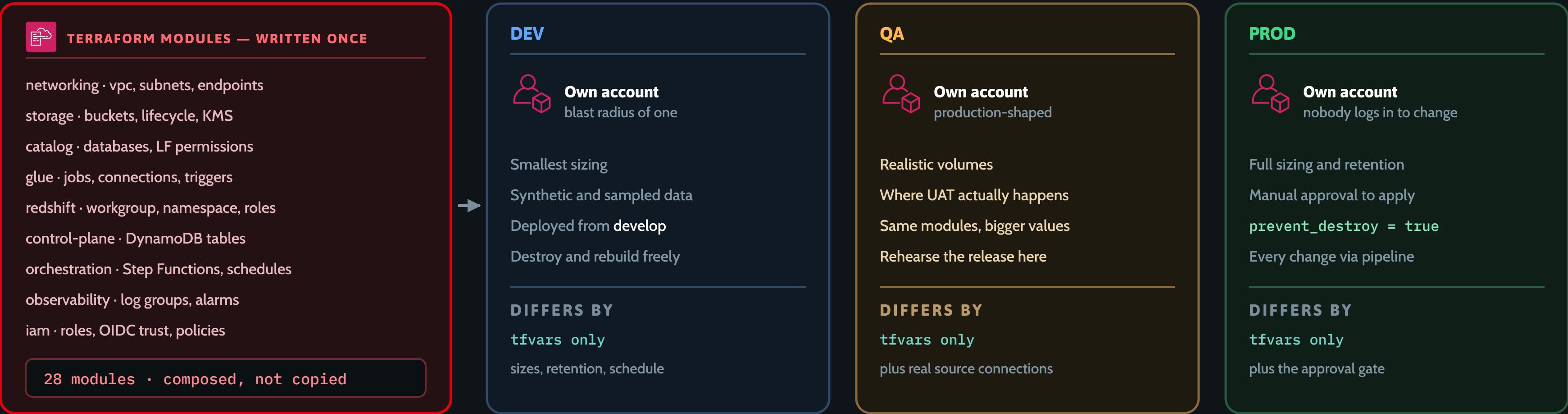
- ☐ I debug downward through the layers, not straight to S3

You've got it when you can...

...be told "the table is there but Redshift cannot see it" and name the two most likely layers before opening anything.

Environments And Composition

One set of modules, composed three times. What differs between environments should be values — never code.



VERIFY THESE THREE IN THE REPO — THEY DRIFT QUIETLY

1. Remote state is genuinely separate per environment · 2. OIDC trust pins each env to the branch that actually deploys it · 3. prevent_destroy is on every stateful prod resource

IN PLAIN ENGLISH

One set of blueprints, three buildings. If dev and prod need different blueprints, one of them is not really testing the other.

D13 · Environments And Composition

Diagram: [_render/D13-environments-terraform.html](#)

What it shows

One set of modules, composed three times. Roughly 28 Terraform modules — networking, storage, catalog, glue, redshift, control-plane, orchestration, observability, iam — instantiated per environment.

What differs between environments should be *values*. Never code.

If dev and prod need different modules, one of them is not really testing the other, and the test you thought you had is imaginary.

The three environments

Dev — its own account, smallest sizing, synthetic and sampled data, deployed from `develop` , destroyed and rebuilt freely.

QA — its own account, production-shaped: realistic volumes, real source connections, where UAT and reconciliation actually happen. Same modules, bigger values.

Prod — its own account. Nobody logs in to change anything. Manual approval before apply, `prevent_destroy = true` on stateful resources, full retention and alarms.

An account each. Not one account with prefixes — the blast radius of a mistake should stop at an account boundary.

Verify these three in the repo

They drift quietly, and each one is invisible until the day it matters:

- 1. Remote state is genuinely separate per environment.** A shared state bucket means a dev mistake can reach prod state. Worth checking the actual `backend.tf` values rather than the comments above them — comments describing a split that no longer exists are the classic trap.
- 2. OIDC trust pins each environment to the branch that actually deploys it.** If dev's trust policy names `main` while the pipeline deploys dev from `develop` , deployments fail confusingly — or worse, succeed from the wrong branch.
- 3. `prevent_destroy` is on every stateful prod resource.** Buckets, catalogs, the Redshift namespace, the control-plane tables.

These are stated as checks rather than findings because they are the kind of thing that regresses. Verify them against the current repo before quoting the state of any of them.

Why "tfvars only"

The value is not tidiness. It is that a QA test **means something**. If QA runs the same modules as prod with different numbers, then QA passing is evidence. If QA runs different code, QA passing is a nice feeling.

Checklist

- ☐ Modules are written once and composed per environment
- ☐ Environments differ by tfvars, not by code
- ☐ One AWS account per environment
- ☐ Remote state verified separate — in the code, not the comments
- ☐ OIDC trust verified against the branch that actually deploys
- ☐ `prevent_destroy` on every stateful prod resource
- ☐ Nobody has console write access to prod

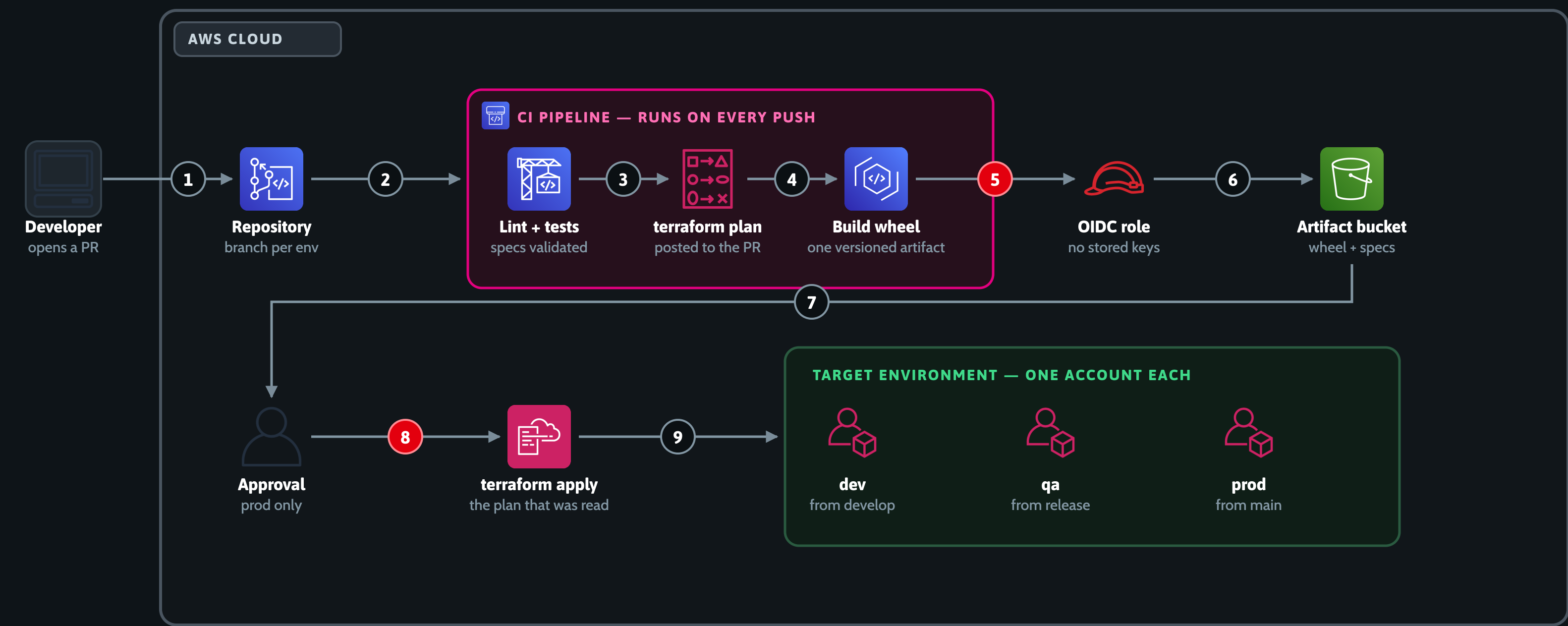
D13 · Environments And Composition

You've got it when you can...

...open the three `backend.tf` files and the OIDC trust policy and say, from the code alone, whether the environments are genuinely isolated.

From Commit To Environment

Nine steps, no long-lived AWS keys anywhere, and one manual gate that only exists in front of production.



IN PLAIN ENGLISH

The pipeline is the only key holder. If a person can change production without going through this picture, the picture is decoration.

D14 · From Commit To Environment

Diagram: [_render/D14-cicd-deployment.html](#)

What this pattern is for

Getting a change from a developer's branch into an environment **with no long-lived AWS credentials anywhere**, and with exactly one manual gate — in front of production only.

The nine steps

- 1 · **A developer opens a pull request.** Not a push to a deploy branch. The PR is where review happens, and review is the only control that scales.
- 2 · **The repository, branch per environment.** `develop` → dev, `release` → qa, `main` → prod. The mapping must match the OIDC trust policies exactly (D13) or deployments fail in ways that take a morning to diagnose.
- 3 · **Lint and tests run.** Including **spec validation** — the per-table declarations are the thing most likely to be wrong, and they are pure data, so they can be checked in seconds without a cluster.
- 4 · **`terraform plan` , posted to the PR.** ★ The plan is the review artefact. A reviewer approves *a specific plan*, not a vague intention. This is what makes "what will this change?" answerable before it changes anything.
- 5 · **OIDC, not stored keys.** ★ The pipeline assumes a role via OpenID Connect and receives short-lived credentials. **There is no AWS access key in the CI system to leak, rotate or forget.** If you take one thing from this diagram, take this one.
- 6 · **The wheel is built once.** One versioned artifact, published to the artifact store. The same bytes are deployed to dev, then qa, then prod — not rebuilt per environment, because a rebuild is a chance to be different.

- 7 · **Artifacts land in a bucket.** The wheel and the specs, versioned. This is what "promotion" actually moves (D20).
- 8 · **Manual approval — production only.** ★ Dev and QA deploy automatically; slowing them down teaches people to route around the pipeline. Production has one gate, and the approver is looking at the plan from step 4.
- 9 · **`terraform apply` — the plan that was reviewed.** Not a fresh plan generated at apply time. Applying an unreviewed plan makes step 4 theatre.

The rule

The pipeline is the only key holder.

If a person can change production without going through this picture, the picture is decoration. That includes emergency fixes — an emergency path that bypasses review is the one that will be used at 2am by someone tired.

What breaks if you skip a piece

- **Stored access keys** — they leak, and they never expire.
- **No plan in the PR** — reviewers approve intentions, not changes.
- **Rebuild per environment** — prod runs bytes nobody tested.
- **Approval gates on dev and QA** — people route around the pipeline.
- **Fresh plan at apply time** — the review was of something else.

D14 · From Commit To Environment

Checklist

- ☐ No long-lived AWS credentials in CI
- ☐ OIDC trust matches the branch that actually deploys each environment
- ☐ `terraform plan` is posted to the PR and reviewed
- ☐ One artifact built once and promoted unchanged
- ☐ Manual approval on production only

- ☐ Apply uses the reviewed plan
- ☐ No human has console write access to production

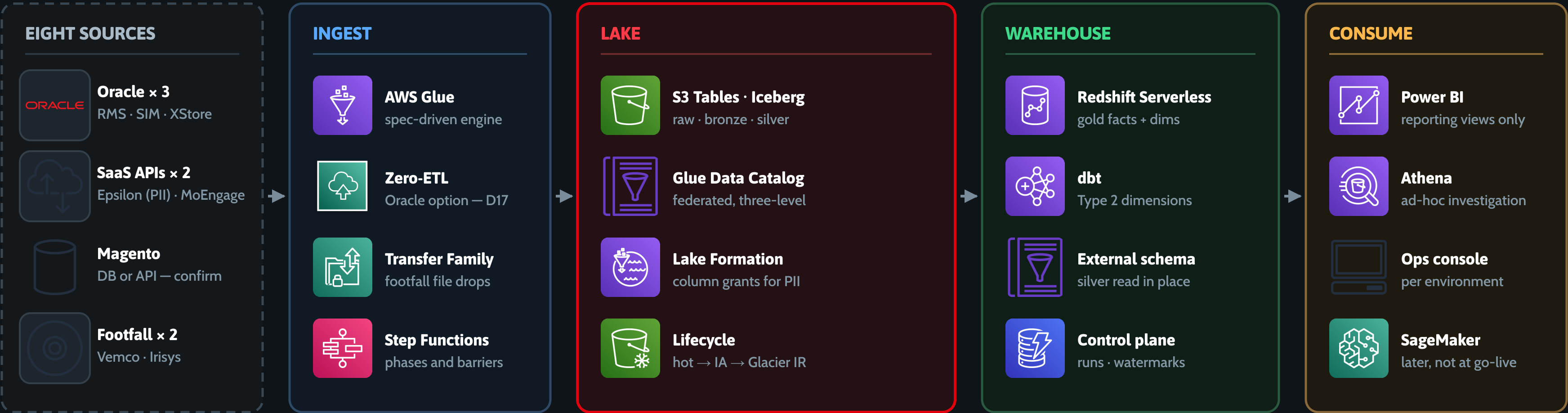
You've got it when you can...

...be asked "how did that get into production?" and answer with a PR number, a plan, an approver and a pipeline run — every time, including for the urgent one.

Apparel Group — Target Architecture

Eight sources, one lakehouse, one warehouse. The same shape as Tamimi, with the lessons already applied.

1 · THE PROPOSED DATA PATH



WHAT WE DO DIFFERENTLY THIS TIME

Engine before pipelines · separate state and OIDC trust per environment from day one · PII masked at the column · the ops console shipped with the platform, not after it

IN PLAIN ENGLISH

Same building, second time. We already know where the damp gets in — so this time the membrane goes in before the plaster.

D15 · Apparel Group — Target Architecture

Diagram: [_render/D15-ag-target-architecture.html](#)

What it shows

Eight sources, one lakehouse, one warehouse. **The same shape as Tamimi (D09), with the lessons already applied.**

This is the diagram that goes in the proposal. It should be recognisable to anyone who has seen D01, because the point is that this is a known pattern being applied deliberately — not an invention.

The eight sources, in five classes

CLASS	SOURCES
Oracle databases	RMS · SIM · XStore
SaaS APIs	Epsilon (PII) · MoEngage
E-commerce	Magento — DB or API, to confirm
File drops	Vemco · Irisys footfall

Five classes, **three connector classes** (D16). The variety lives in the specs.

What we do differently this time

Four things, stated plainly because they are the value of having done this before:

- Engine before pipelines.** The spec-driven engine exists before source one, not retrofitted at source five.
- Separate state and OIDC trust per environment, from day one** — verified in code, not assumed from comments (D13).
- PII masked at the column**, never by keeping a filtered second copy (D19).

- The ops console ships with the platform**, not after go-live (D26). It reads the control plane, which exists anyway.

The open decisions this diagram does not settle

Deliberately, because they need evidence rather than a picture:

- How the three Oracle sources are ingested** — Glue JDBC, DMS CDC or zero-ETL (D17). Worth a measured spike on one source.
- Whether Magento is a database or an API.** Affects which connector class it joins.
- Whether SageMaker is in scope at go-live.** Drawn, but marked "later" — it is a common source of scope creep.

What is not on this diagram

Streaming. Nothing in the SOW requires it, and drawing it would invite a conversation that costs money and delivers nothing (D24). If it becomes a requirement, it is an addition, not a redesign.

Checklist

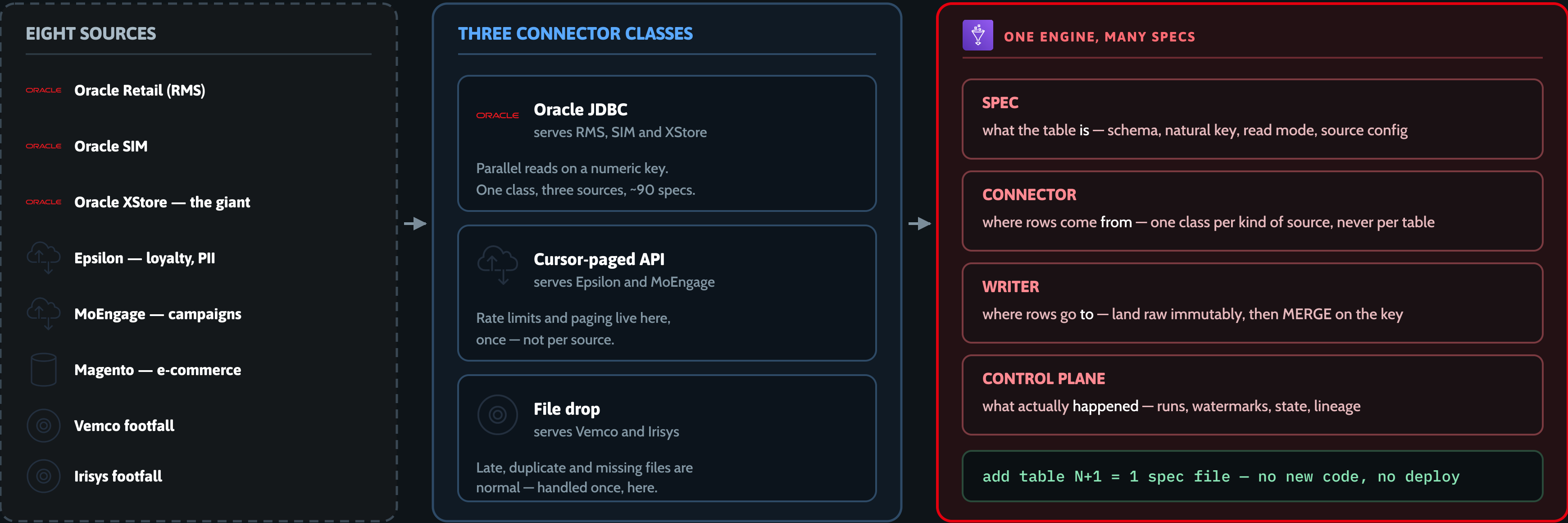
- ☐ I can name the eight sources and their five classes
- ☐ I can name the four things we are doing differently
- ☐ I know which decisions this diagram deliberately leaves open
- ☐ I know why streaming is absent
- ☐ I can present this to a client sponsor in five minutes

You've got it when you can...

...show this alongside D09 and make the case that we are applying a proven pattern rather than experimenting on their money.

Eight Sources, Three Connectors

The variety lives in the specs, not in the code. That is what makes the ninth source a config change rather than a project.



IN PLAIN ENGLISH

Build the printer once, not a hundred hand-written letters. Every new table after that is just another page you feed it.

D16 · Eight Sources, Three Connectors

Diagram: [_render/D16-source-onboarding.html](#)

What it shows

Eight sources collapsing into **three connector classes**, feeding one engine driven by specs.

The variety lives in the specs, not in the code.

That single property is what makes the ninth source a config change rather than a project.

The three connector classes

Oracle JDBC — serves RMS, SIM and XStore. Parallel reads on a numeric key. One class, three sources, roughly ninety specs. Sizing differs per source (XStore is the giant), but sizing is a spec field, not a different class.

Cursor-paged API — serves Epsilon and MoEngage. Rate limits, paging and retry live here **once**, not re-solved per source. This is where the SaaS engineering problem actually is.

File drop — serves Vemco and Irisys. Late, duplicate and missing files are normal operating conditions, handled once, here.

Magento joins whichever of the first two it turns out to be — a question to settle in discovery, not in code.

The four seams

SEAM	OWNS
Spec	what the table <i>is</i> — schema, natural key, read mode, source config
Connector	where rows come <i>from</i> — one class per kind of source, never per table
Writer	where rows go <i>to</i> — land raw immutably, then MERGE on the key
Control plane	what actually <i>happened</i> — runs, watermarks, state, lineage

Jobs only wire the seams together. If a job file contains business logic, that logic belongs behind a seam.

The acceptance test

```
add table N+1 = 1 spec file — no new code, no deploy
```

Check this at source two, not source eight. Onboard the second source and count the lines of new Python. If it is not close to zero, a seam is in the wrong place — fix it while there are two, not while there are eight.

Why this beats eight pipelines

- A fix lands **once** and every table gets it on the next run.
- A YAML spec **diffs in a pull request**, and a non-Python reviewer can read it.
- Spec parsing is a pure function, so the thing most likely to be wrong is **testable without a cluster**.
- "How do I add a table?" has a three-line answer a new joiner can execute on day two.

D16 · Eight Sources, Three Connectors

Checklist

- ☐ Eight sources map to three connector classes
- ☐ The spec model is typed and validated before compute starts
- ☐ The four seams exist as separate modules
- ☐ No job file contains a source name or an `if source == ...`

- ☐ Adding source two required zero new Python — verified, not assumed
- ☐ Routing and schedule live in the control plane, not in specs

You've got it when you can...

...be handed a ninth source and estimate it as "one connector class if it is new, otherwise one spec per table" — and defend that by naming which seam each piece of work lands in.

Oracle — Three Ways In

Three of the eight sources are Oracle. This is a real decision with real arguments, and it is worth a spike before committing.

1 · GLUE JDBC

you build it

ORACLE

→



→



Oracle

Glue

S3 / Iceberg

Full transformation control

Lands in the lake — medallion intact

Cheapest per row at XStore volume

You build it, and you operate it

DEFAULT FOR ALL THREE

unless the spike says otherwise

2 · DMS · CDC

reads the log

ORACLE

→



→



Oracle redo

DMS

S3 / Redshift

Catches hard deletes — the decider

Minutes, not hours

Almost no query load on the source

Still a pipeline to operate

IF DELETES MATTER

ask this of every table, week one

3 · ZERO-ETL

no pipeline

ORACLE

→



→



Oracle

managed

Redshift only

No pipeline to build or operate

Self-managed → Redshift only

No transform in flight at all

Ongoing cost on a huge table

SPIKE IT FIRST

one source, measured, before we commit

DECIDE PER SOURCE, NOT ONCE FOR THE PLATFORM

XStore's volume argues for Glue. A small, slow-changing reference table argues for zero-ETL — or for not ingesting it at all and mounting it as a federated catalog (DO6).

IN PLAIN ENGLISH

Own a van, hire a courier, or have it delivered. The right answer depends on the parcel — and you are shipping three very different parcels.

D17 · Oracle — Three Ways In

Diagram: [_render/D17-oracle-three-options.html](#)

Why this diagram exists

Three of Apparel Group's eight sources are Oracle, and they are the three that carry the volume. How we ingest them is the single largest technical decision in the programme.

It is a real decision with real arguments on both sides — not a preference, and not something to settle in a slide.

Option 1 · Glue JDBC

Oracle → Glue → S3 / Iceberg

- Full transformation control
- Lands in the lake — medallion intact
- Cheapest per row at XStore volume
- ⚠️ You build it, and you operate it

Verdict: the default for all three, unless the spike says otherwise.

Option 2 · DMS · CDC

Oracle redo → DMS → S3 / Redshift

- Catches hard deletes — usually the decider
- Minutes, not hours
- Almost no query load on the source
- ⚠️ Still a pipeline to operate, plus compaction (D22)

Verdict: if deletes matter. Ask of every table, in week one, of someone who knows.

Option 3 · Zero-ETL

Oracle → managed integration → Redshift **only**

- No pipeline to build or operate
- ⚠️ Self-managed sources can only target Redshift — not S3, not S3 Tables, not RMS
- ⚠️ No transformation in flight at all
- ⚠️ Ongoing cost on a very large, high-churn table

Verdict: spike it first. One source, measured, before committing.

The rule

Decide per source, not once for the platform.

XStore's volume argues for Glue. A small, slow-changing reference table argues for zero-ETL — or for not ingesting it at all and **mounting it as a federated catalog** (DO6), which is the option people forget entirely.

The two questions that decide it

1. **Does this source hard-delete rows?** If yes, a watermarked batch pull will drift wrong and never tell you. That points at CDC or zero-ETL.
2. **Does our architecture need it in the lake first?** If yes, zero-ETL is out for a self-managed source, because it can only reach Redshift. That constraint alone eliminates option 3 for anything that must flow through bronze and silver.

Those two questions, asked per source, resolve most of the matrix before anyone runs a benchmark.

D17 · Oracle — Three Ways In

What the spike should measure

Not "does it work" — it will. Measure:

- **Throughput** on the largest table, and what it costs
- **Ongoing cost** at realistic change rates, over a week not an hour
- **Operational surface** — what breaks, how you would know, how you would fix it
- **Where the transformation lands**, and who owns it there

Checklist

- ☐ I can present all three options with their real limits

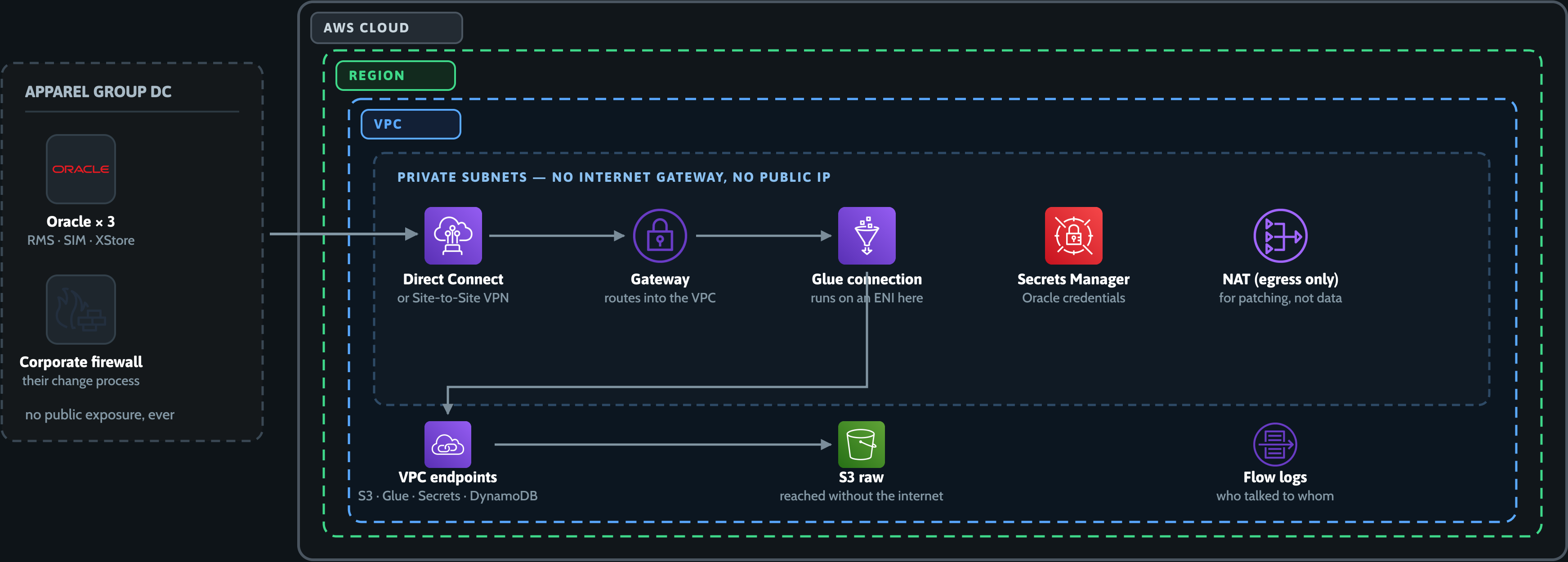
- ☐ I know self-managed zero-ETL targets Redshift only
- ☐ I have asked the hard-delete question for every Oracle table
- ☐ I know federated catalog mounting is a fourth option for reference data
- ☐ The decision is per source, and written down with its reason
- ☐ A spike is planned before commitment, with what it measures defined

You've got it when you can...

...be pushed to "just pick one for all three Oracle sources" and explain, with the two questions, why that would be the wrong shape of decision.

Reaching The On-Prem Sources

The network path is procured, not coded — and it has a lead time. Raise it in week one or it becomes the critical path in week nine.



IN PLAIN ENGLISH

A private road between two buildings, not a public one with a gate. Nothing here is reachable from the internet — including us.

D18 · Reaching The On-Prem Sources

Diagram: [_render/D18-network-connectivity.html](#)

What it shows

How Glue reaches three Oracle databases sitting in Apparel Group's data centre, with **nothing exposed to the internet in either direction**.

The network path is procured, not coded. It has weeks of lead time. Raise it in week one or it becomes the critical path in week nine.

That sentence is the reason this diagram exists at all. It is the most common cause of slippage on projects of this shape, and it is entirely predictable.

The path

Their side. Three Oracle databases behind a corporate firewall, governed by their change process — which is another lead time, and another team's calendar.

The link. Direct Connect or Site-to-Site VPN into a gateway that routes into our VPC.

Private subnets. No internet gateway, no public IPs. The **Glue connection runs on an ENI inside the subnet**, which is what lets a serverless job reach a private database at all — and it is the piece people are surprised by.

Credentials. Secrets Manager, never in job parameters or a spec file.

Egress. A NAT gateway for patching only — not a data path.

VPC endpoints. S3, Glue, Secrets Manager and DynamoDB reached **without traversing the internet**. This matters for both security posture and data-transfer cost (D29).

Flow logs. So "who talked to whom" is answerable afterwards.

The three things to start in week one

1. **The network request itself** — the long pole, and not ours to expedite.
2. **Firewall rules on their side** — their change process, their calendar.
3. **A test credential and a single reachable table** — so that the day the link comes up, you can prove it in ten minutes rather than starting to debug.

That third one is the trick worth remembering. Teams wait for the link, then spend a week discovering credentials are wrong.

Why no public path, ever

Not policy for its own sake. A publicly reachable database is a database that will eventually be reached by someone else, and the discussion after that is not a technical one.

Private subnets plus endpoints also mean traffic to S3 and Glue never leaves the AWS network, which is cheaper as well as safer.

Checklist

- ☐ Network request raised in week one, with a named owner on their side
- ☐ Glue connection configured with subnet and security group
- ☐ Credentials in Secrets Manager, referenced not embedded
- ☐ VPC endpoints for S3, Glue, Secrets Manager, DynamoDB
- ☐ No public IPs and no internet gateway on the data subnets
- ☐ NAT is egress-only, and not on the data path
- ☐ Flow logs enabled
- ☐ A single table proven reachable the day the link is up

D18 · Reaching The On-Prem Sources

You've got it when you can...

...sit in a kickoff and identify the network path as the critical path before anyone has written a line of code — and name the three things to start that morning.

PII And The Security Model

Epsilon carries customer data. Four layers stand between it and someone who should not see it — and none of them is a filtered copy.

1 · FOUR LAYERS, OUTSIDE IN

1 · IDENTITY



IAM roles
no user has a password



Identity Center
federated login only



OIDC for CI
no long-lived keys

THE RULE

Grant to roles, never to people.
A leaver is removed from a role,
not hunted through grants.

WHO YOU ARE

2 · ENCRYPTION



KMS
a key per bucket



Secrets Manager
source credentials



TLS everywhere
in transit, no exceptions

THE RULE

Bucket policy denies any
unencrypted write, so nobody
has to remember.

AT REST AND IN TRANSIT

3 · DATA ACCESS



Lake Formation
column-level grants



RLS and CLS
inside the warehouse



Macie
finds PII you missed

THE RULE

Mask at the column, never by
keeping a second filtered copy.
A copy is a second thing to leak.

WHAT YOU MAY SEE

4 · AUDIT



CloudTrail
who did what, when



GuardDuty
unusual behaviour



AWS Config
has it drifted?

THE RULE

Every grant is Terraform, so
access has an author, a reason
and a reviewer.

PROVE IT LATER

IN PLAIN ENGLISH

Four locks, not one big one. The important one is the third: people get a view of the same table, never their own private copy of it.

D19 · PII And The Security Model

Diagram: [_render/D19-pii-security.html](#)

What it shows

Four layers between customer data and someone who should not see it — and **none of them is a filtered copy**.

Epsilon carries PII. That single fact drives most of the design decisions on this diagram.

The four layers

1 · Identity — *who you are*

IAM roles, Identity Center for federated login, OIDC for CI. **No user has a database password.** *The rule:* grant to roles, never to people. A leaver is removed from a role, not hunted through scattered grants.

2 · Encryption — *at rest and in transit*

KMS with a key per bucket, Secrets Manager for source credentials, TLS everywhere. *The rule:* the bucket policy **denies any unencrypted write**, so nobody has to remember.

3 · Data access — *what you may see*

Lake Formation column-level grants, RLS and CLS inside Redshift, Macie scanning for PII nobody classified. *The rule:* **mask at the column, never by keeping a second filtered copy**. A copy is a second thing to secure, a second thing to keep in sync, and a second thing to leak.

4 · Audit — *prove it later*

CloudTrail records who did what. GuardDuty flags unusual behaviour. Config detects drift. *The rule:* every grant is Terraform, so access has an author, a reason and a reviewer.

The layer that actually matters

Layer 3. The other three are hygiene that any competent platform has. Layer 3 is where teams take the shortcut — building a "safe" copy of the customer table with the sensitive columns dropped, and handing that out.

That shortcut fails in three ways: the copy drifts, the copy has to be secured separately, and the moment someone needs one more column the whole thing is rebuilt. Column-level grants against **one** table have none of those problems.

Test it from the right seat

Verify masking from a real consumer role, not from an admin session. An admin sees everything, which makes admin a useless place to test from — and it is exactly where most people test.

The KMS trap

A role can hold a perfectly good Lake Formation grant on a table and still get access denied, because it lacks **decrypt permission on the KMS key**. It is the single most confusing failure in the stack (D28), and it is worth knowing before you meet it at speed.

D19 · PII And The Security Model

Checklist

- ☐ No database passwords anywhere
- ☐ Grants go to roles, never to individuals
- ☐ Bucket policy denies unencrypted writes
- ☐ PII masked with column grants, not a filtered copy
- ☐ Roles have **KMS decrypt** as well as the table grant

- ☐ CloudTrail on, retained long enough for an audit
- ☐ Every grant in Terraform
- ☐ Masking tested from a consumer role, not an admin one

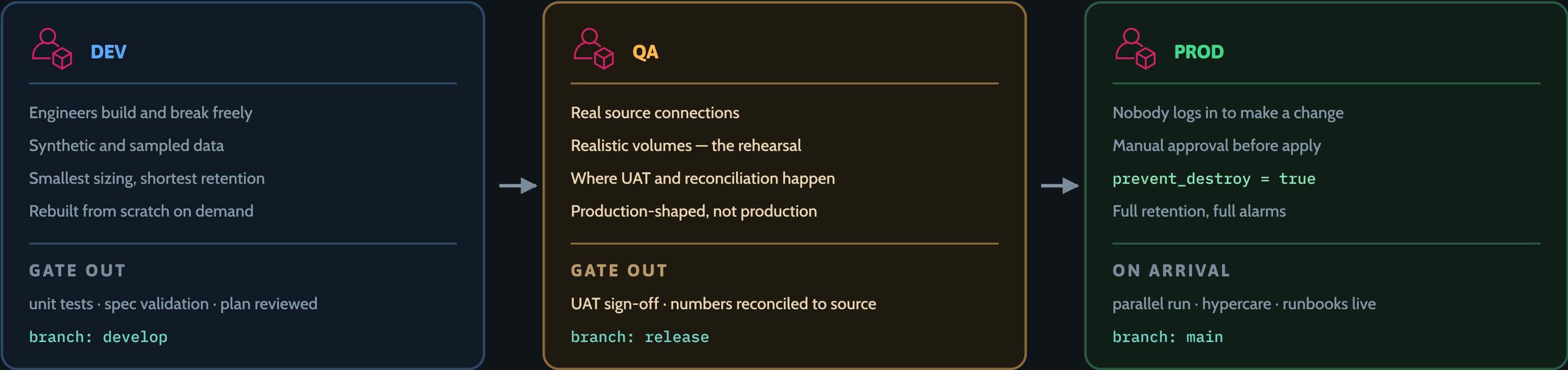
You've got it when you can...

...be asked for "a version of the customer table without the personal data" and set up a column-level grant instead — then show the requester it works from their own login.

What Gets Promoted, And What Never Does

Code and config move forward through the environments. Data, credentials and manual fixes do not move at all.

1 · THE PROMOTION PATH



WHAT MOVES FORWARD

- Terraform modules and the composed stacks
- The engine wheel, versioned, built once
- Source specs, dbt models, dashboards

WHAT NEVER MOVES

- Data — each environment loads its own
- Credentials — a separate secret per account
- Manual fixes — if it was not in the plan, it did not happen

IN PLAIN ENGLISH

The recipe travels between kitchens. The ingredients and the keys stay where they are — and nobody is allowed to season production by hand.

D20 · What Gets Promoted

Diagram: [_render/D20-promotion-path.html](#)

What it shows

Code and configuration move forward. Data, credentials and manual fixes do not move at all.

Most environment confusion comes from being vague about which of those two lists a thing belongs to.

What moves forward

- Terraform modules and the composed stacks
- The engine wheel — versioned, **built once** (D14)
- Source specs, dbt models, dashboard definitions

What never moves

- **Data.** Each environment loads its own. Copying prod data into dev is how PII ends up somewhere it is not governed.
- **Credentials.** A separate secret per account. A credential that works in two environments is a credential that will be used in the wrong one.
- **Manual fixes.** If it was not in the plan, it did not happen — and it will be gone at the next apply, usually at the worst moment.

The gates

Dev → **QA**: unit tests, spec validation, plan reviewed. **QA** → **Prod**: UAT sign-off **and numbers reconciled to source**.

That second gate is the one worth defending. "It runs" is not the same as "it is right", and reconciliation is the only evidence that separates them.

Why QA has to be production-shaped

If QA runs different code, or a tenth of the data, or without the real source connections, then QA passing is a nice feeling rather than evidence. Same modules, bigger values, real connections — that is what makes the QA gate mean something (D13).

The rule about manual fixes

Worth being blunt, because everyone is tempted at least once:

A change made by hand in production is a change that will vanish silently at the next apply — and nobody will connect the outage to it.

The correct response to an urgent production problem is an urgent pull request, not an urgent console session. If the pipeline is too slow to allow that, fix the pipeline; do not build a habit of bypassing it.

Checklist

- ☐ Code and config promote; data and credentials do not
- ☐ One artifact built once, promoted unchanged
- ☐ QA has real source connections and realistic volumes
- ☐ The QA gate includes reconciliation, not only "it ran"
- ☐ Production changes only ever arrive through the pipeline
- ☐ A separate secret per environment
- ☐ Nobody has console write access to production

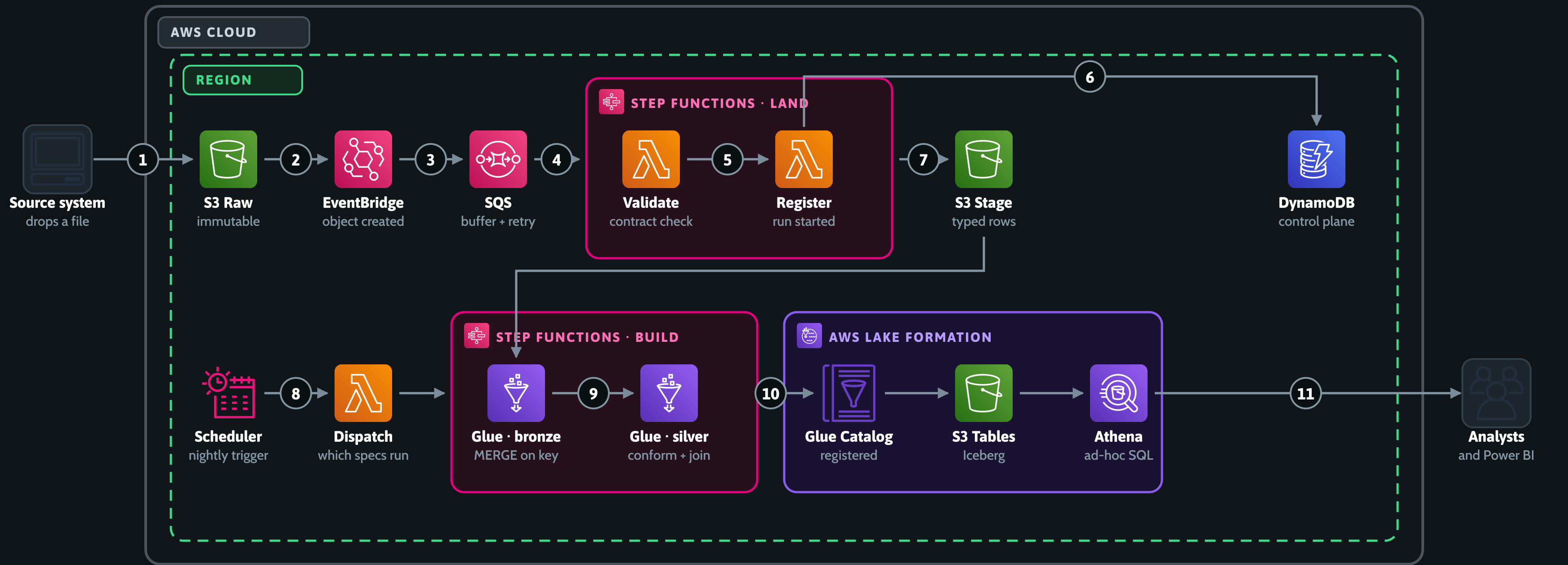
D20 · What Gets Promoted

You've got it when you can...

...be asked to "just fix it in prod, we'll do the PR after" and offer the faster correct path instead — because you already know how long the pipeline takes.

Event-Driven File Ingestion

A file lands and the platform reacts. Eleven steps, no polling, nothing running when there is nothing to do.



IN PLAIN ENGLISH

Nobody stands at the door checking for deliveries. The doorbell wakes the right person, and everyone else stays asleep — and unbilled.

D21 · Event-Driven File Ingestion

Diagram: [_render/D21-event-driven-ingestion.html](#)

What this pattern is for

A source system drops a file and the platform reacts. **No polling, nothing running when there is nothing to do, and no server anyone has to patch.**

Use it when arrival is unpredictable — SFTP drops, partner feeds, footfall exports — and you want the cost to be zero on a day nothing arrives. The alternative (a scheduled job that looks for files) burns compute on every empty check and still adds latency on a busy day.

The eleven steps

- 1 · **A file lands in the raw bucket.** The source system writes to `raw/<source>/<table>/dt=.../` . That is the only thing it has to do, and the only interface we ask it to honour. The object is never edited afterwards.
- 2 · **S3 raises an object-created event to EventBridge.** S3 notifies EventBridge rather than calling a Lambda directly. This matters: EventBridge lets several independent rules react to the same object without the producer knowing about any of them, so adding a consumer later is a rule, not a change.
- 3 · **EventBridge puts a message on SQS.** The queue is the shock absorber. A hundred files arriving at once become a hundred queued messages instead of a hundred concurrent executions, and a failure gets retried with backoff rather than lost. Give it a dead-letter queue on day one.
- 4 · **SQS triggers the landing Step Functions workflow.** State machine, not a chain of Lambdas calling each other. Retries, error paths and timeouts are declared rather than coded, and a failed execution is visible in the

console with the exact step that failed.

- 5 · **Validate, then register.** The first Lambda checks the file against the declared contract — expected columns, types, partition, non-empty. A file that fails is moved aside and alarmed, **not** processed into the lake. The second Lambda records that a run has started.
- 6 · **The run is written to the control plane.** DynamoDB holds `runs` , `watermarks` , `pipeline-state` and `lineage` . This is what makes "did last night work?" a query rather than a log search, and what the ops console reads.
- 7 · **Typed rows land in the stage bucket.** Raw stays raw, forever. The staged copy is typed and partitioned, which is what the Glue jobs read. Two buckets, two jobs: one preserves evidence, the other is convenient to process.
- 8 · **On the schedule, a dispatcher decides what runs.** EventBridge Scheduler triggers a dispatch Lambda that reads the control plane and works out **which specs are due** — not a hardcoded list. This is what makes adding a source a config change rather than a deployment.
- 9 · **Glue builds bronze, then silver.** Bronze does a `MERGE` on the natural key, so re-running is safe. Silver conforms and joins. Both are Iceberg tables, so both support row-level updates and time travel.
- 10 · **Tables are registered in the catalog, inside the Lake Formation scope.** Registration is **declarative** — we do not run crawlers on a schedule and let inferred schema drift silently. From this point, Lake Formation governs who can read what, at column level, for every engine.
- 11 · **Analysts query through Athena.** Serverless SQL, priced per byte scanned, with a workgroup limit set so a bad query is an error rather than an invoice. Power BI reads the modelled layer; Athena is for the questions nobody anticipated.

D21 · Event-Driven File Ingestion

Why it is shaped this way

DECISION	REASON
S3 → EventBridge , not S3 → Lambda	many consumers can react without the producer changing
An SQS queue in the middle	absorbs bursts, retries with backoff, dead-letters poison messages
Step Functions , not chained Lambdas	retries and error paths are declared and visible
Two buckets (raw, stage)	raw is evidence; stage is convenience. Never conflate them
Control plane in DynamoDB	run state is queryable, not buried in CloudWatch Logs
A dispatcher reading config	adding a source is a spec file, not a deploy

What breaks if you skip a piece

- **No queue** — a burst of files becomes a burst of concurrent executions, and throttling failures are silent.
- **No validation step** — a malformed file becomes a malformed table, discovered downstream in a report.
- **No control plane** — every "did it run?" question becomes a log search, and the ops console has nothing to read.
- **Crawlers instead of declarative registration** — a schema change upstream silently changes your table definition.

On Apparel Group

This is exactly the shape for **Vemco** and **Irisys** footfall feeds: small per-store files, unpredictable arrival, and long gaps where nothing comes. Polling those on a schedule would cost more than the data is worth.

It also handles the failure mode those feeds actually have — **arriving twice**. Step 9's **MERGE** on the natural key makes a duplicate delivery a no-op rather than a doubled figure.

The three Oracle sources take a different path (D17, D22, D23); the SaaS APIs a third (D16). This diagram is one of several ingestion shapes, not the only one.

Checklist

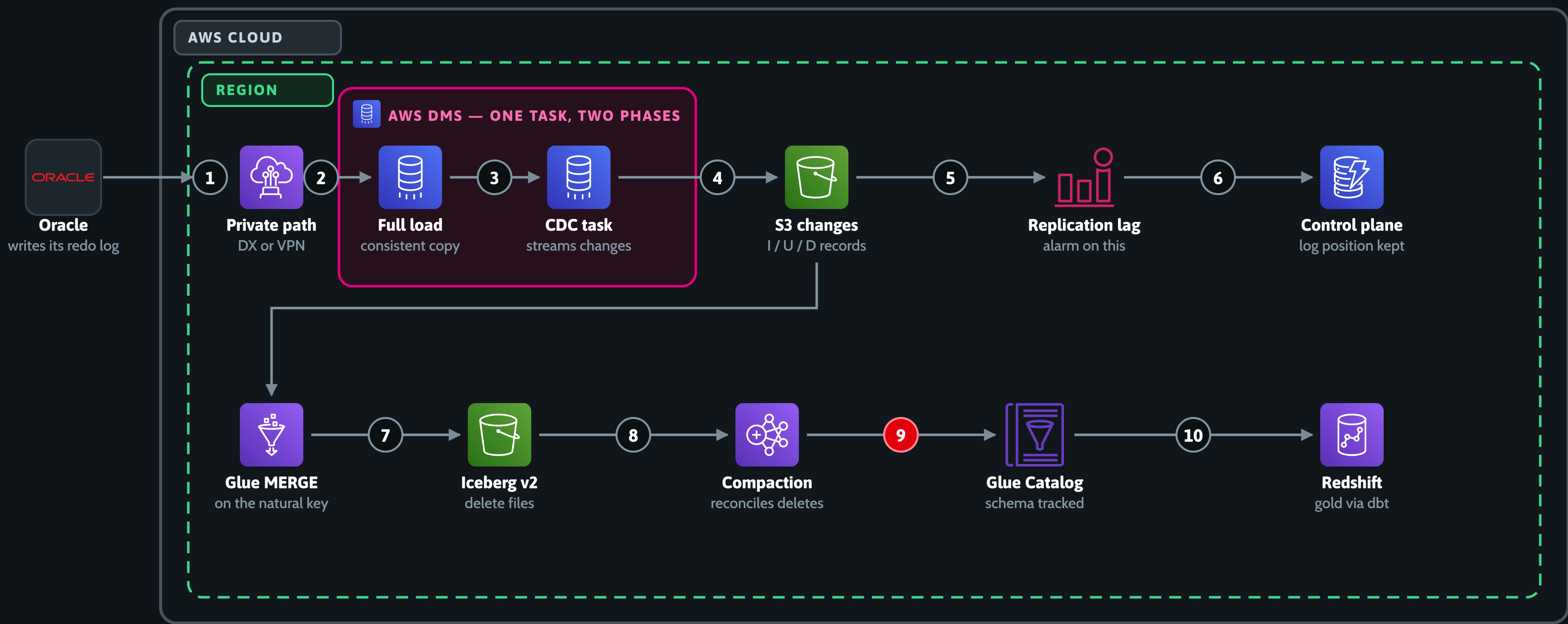
- ☐ S3 notifies EventBridge, not a Lambda directly
- ☐ There is a queue, and it has a dead-letter queue
- ☐ Validation happens before anything reaches a table
- ☐ Run state is written to the control plane, not just logged
- ☐ Raw and stage are separate buckets with different lifecycles
- ☐ Dispatch reads config; adding a source needs no deployment
- ☐ Glue MERGEs on a key, so a duplicate file is harmless
- ☐ Catalog registration is declarative, not a scheduled crawler
- ☐ The Athena workgroup has a per-query scan limit

You've got it when you can...

...draw this on a whiteboard from memory, and for each of the eleven steps say what happens **when that step fails** — because that is the question a client actually asks.

CDC — Reading The Log, Not The Table

Ten steps. Step nine is the whole reason to choose CDC: a deleted row actually disappears.



IN PLAIN ENGLISH

Do not count the shelves every night. Read the till roll — it already lists every sale, every refund, and every item taken back off the shelf.

D22 · CDC — Reading The Log, Not The Table

Diagram: [_render/D22-cdc-pipeline.html](#)

What this pattern is for

Continuous replication out of a relational source, with **almost no query load on that source** and — the part that decides it — **deletes that actually propagate**.

Choose it when the source hard-deletes rows, when you need minutes rather than hours, or when the source DBA will not tolerate a nightly full-table scan.

The ten steps

- 1 · **Oracle writes its redo log**. Every relational engine already logs every change so it can recover from a crash. CDC does not ask the database for anything new — it reads a file the database was writing anyway.
- 2 · **A private network path**. Direct Connect or Site-to-Site VPN. This is procurement, not code, and it has weeks of lead time. It is on the critical path from day one (D18).
- 3 · **DMS runs a full load**. One task, first phase: copy the table as it stands, and record the log position at which that copy was consistent. You do not coordinate this handover yourself — that is the point of using one task.
- 4 · **The same task switches to CDC**. From the recorded position onward, DMS streams every insert, update and delete. There is no gap between the two phases, which is exactly the bug you would introduce hand-rolling this.
- 5 · **Change records land on S3**. Not the current state of the table — a stream of *change events*, each tagged I, U or D. Keep them; they are the audit trail of how the table got to where it is.

6 · **Replication lag is the metric**. Not "did the task run" — **how far behind is it**. A CDC task that is running but four hours behind is failing silently, and only this metric shows it. Alarm on it before go-live, not after the first incident.

7 · **Glue MERGEs into Iceberg**. Insert, update and delete applied against the natural key. Because it is a MERGE, replaying the same change file twice is harmless — which you will need on the day the task restarts.

8 · **Iceberg v2 writes delete files**. A deleted row is not removed from the data file; a small delete file records it and readers apply it. Fast writes, slightly slower reads.

9 · **Compaction reconciles them**. ★ This is the step teams forget. Without scheduled compaction the delete files accumulate, every read gets slower, and storage grows for data that is logically gone. **CDC without compaction degrades quietly for months**.

10 · **Redshift builds gold**. dbt reads the Iceberg tables and the deleted rows are correctly absent — which is the whole reason you chose CDC over a watermarked pull.

Against the alternative

Module 2 teaches watermarked batch pulls, and for many sources that is the right, simpler answer. The question that separates them:

"Does this source ever hard-delete a row?"

Ask it of someone who knows, of every relational source, on day one. A timestamp-based pull asks "*what changed since?*" — and a deleted row does not answer, because it is simply absent. It stays in your warehouse forever, and nothing tells you.

D22 · CDC — Reading The Log, Not The Table

What breaks if you skip a piece

- **No lag alarm** — the pipeline is "green" and four hours stale.
- **No compaction** — reads slow month over month with no obvious cause.
- **No schema-drift policy** — a new upstream column becomes a 6am surprise instead of a decision.
- **Append instead of MERGE** — a task restart doubles the day.

On Apparel Group

This is the credible alternative to Glue JDBC for **Oracle SIM and XStore**, and D17 sets the three options side by side. The deciding evidence is the answer to step-9's question and the hard-delete question — not a preference.

Checklist

- ☐ Confirmed whether the source hard-deletes, from someone who knows

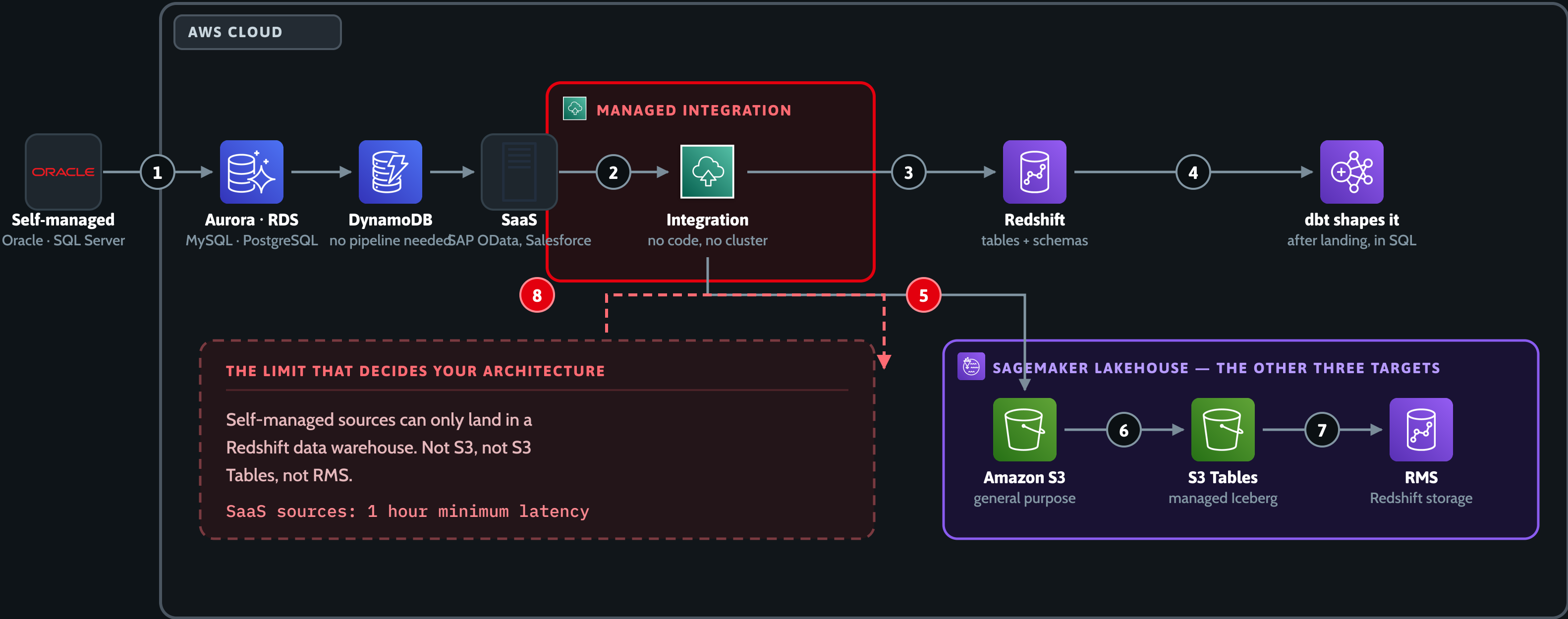
- ☐ Network path requested in week one
- ☐ Full load and CDC are one task, not two
- ☐ Replication **lag** is alarmed, not just task state
- ☐ Loads MERGE on the natural key
- ☐ Compaction is scheduled and monitored
- ☐ Schema-drift policy decided before drift happens

You've got it when you can...

...be shown a "healthy" CDC pipeline and find the two things that make it unhealthy anyway — the lag it is not alarming on, and the compaction nobody scheduled.

Zero-ETL — And Where It Stops

Eight steps, no pipeline. The important part of this diagram is the red arrow that does not exist.



IN PLAIN ENGLISH

A pipe somebody else maintains. You give up the right to change the water on the way through — and you stop being paged about the pipe.

D23 · Zero-ETL — And Where It Stops

Diagram: [_render/D23-zero-etl-deep-dive.html](#)

What this pattern is for

Getting a source table into an analytical target **with no pipeline at all** — no job, no cluster, no code, nothing to schedule. You configure an integration; AWS keeps the two in step.

The most important thing on the diagram is the **red dashed arrow that stops short**. Knowing where zero-ETL cannot reach is what makes it usable rather than a trap.

The eight steps

1 · Pick a source. Three families: AWS databases (Aurora MySQL/PostgreSQL, RDS MySQL, DynamoDB, Oracle Database@AWS), self-managed databases via DMS (Oracle, SQL Server, MySQL, PostgreSQL), and SaaS applications (Salesforce, SAP OData, ServiceNow, Zendesk, Zoho, Meta Ads).

2 · Configure the integration. Source authorisation on one side, an integration on the other. That is the whole operational surface — there is no code artefact to version and no cluster to size.

3 · Data and schemas replicate. Not just rows: **the schema comes too**. New columns appear without you doing anything, which is convenient and is also something to be aware of before it surprises a downstream model.

4 · It lands in Redshift, source-shaped. Column names, types and quirks exactly as the source has them. Zero-ETL replicates; it does not transform.

5 · The limit that decides your architecture. ★ **Self-managed database sources can only target a Redshift data warehouse** — not S3, not S3 Tables, not RMS. If your architecture lands everything in the lake first and

models afterwards, this constraint may rule the option out for exactly the sources you most wanted it for.

6 · AWS-native and SaaS sources have more targets. Those can land in S3, S3 Tables or Redshift Managed Storage via the SageMaker Lakehouse architecture — so the same feature behaves quite differently depending on which family your source is in.

7 · Shaping happens after landing, in SQL. The transformation work does not disappear; it moves. Instead of Spark in the middle it becomes dbt on the other side. That may be a better fit for your team — it is a different place to put the work, not less work.

8 · SaaS sources have a one-hour floor. If someone wrote "near real-time" against a Salesforce or SAP feed, this is the number to put in front of them before the design is signed.

When to choose it

Choose zero-ETL when the source shape is close enough to useful, the target is Redshift, and the value of never operating a pipeline outweighs the loss of in-flight control.

Do not choose it when you need medallion layering in the lake for a self-managed source, when the volume makes continuous replication expensive, or when heavy business logic has to run during ingestion.

On Apparel Group

Three of eight sources are Oracle, and self-managed Oracle → Redshift is supported. That makes this a **real** alternative to hand-building Glue pipelines — see D17 for the three-way comparison.

D23 · Zero-ETL — And Where It Stops

The honest recommendation is a **spike on one Oracle source, measured**, before committing either way. XStore's volume argues for Glue; a small slow-changing reference table argues for zero-ETL, or for not ingesting it at all and mounting it as a federated catalog (DO6).

Checklist

- ☐ I know which of the three source families each source belongs to
- ☐ I know the target restriction for self-managed sources

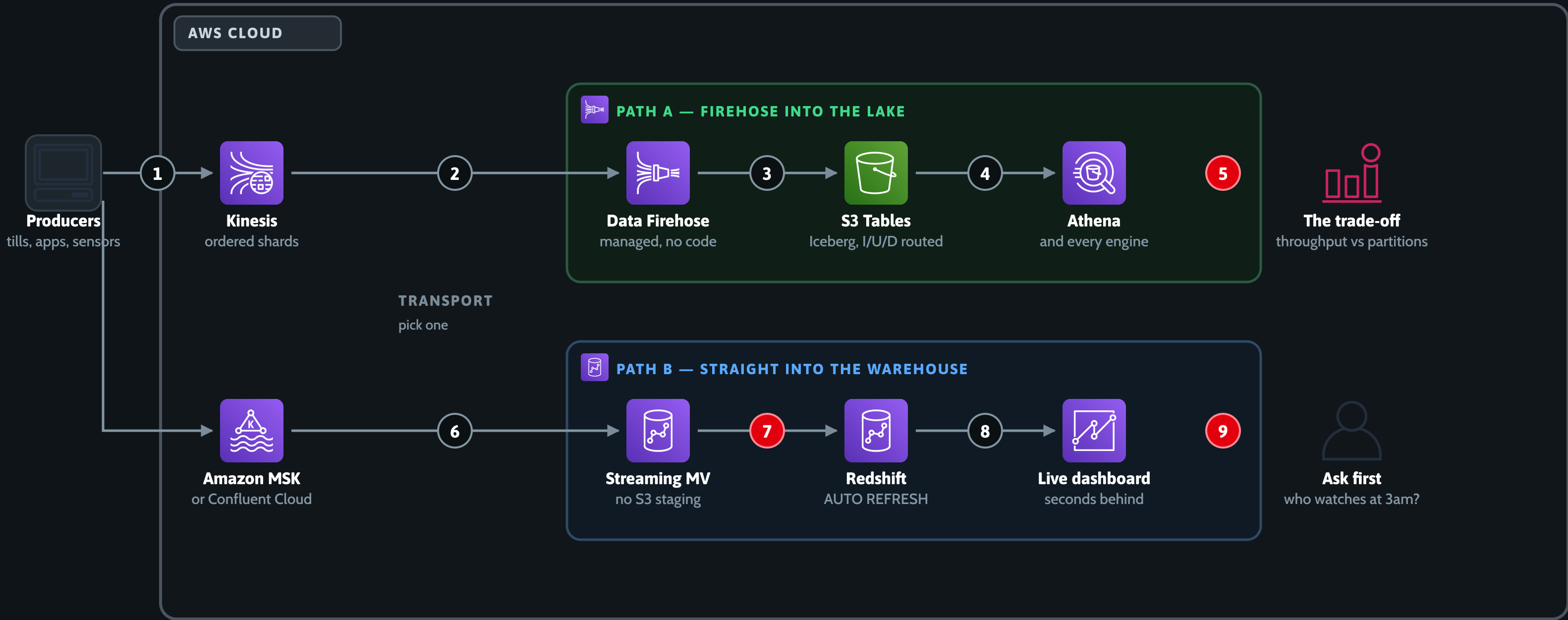
- ☐ I have stated the SaaS latency floor to whoever wrote the requirement
- ☐ I have decided where transformation happens, and who owns it
- ☐ The choice was made per source, not once for the platform

You've got it when you can...

...hear "let's just use zero-ETL for Oracle" and immediately name the one constraint that decides whether that is even possible for our architecture.

Streaming — Two Landing Paths

Same transport, two very different destinations. One goes through S3; the other does not touch it at all.



IN PLAIN ENGLISH

Path A keeps the evidence and feeds everything. Path B is faster and feeds one thing. Most projects want A and think they want B.

D24 · Streaming — Two Landing Paths

Diagram: [_render/D24-streaming-ingestion.html](#)

What this pattern is for

Genuinely event-driven data, where seconds change a decision somebody is making. **Most retail analytics does not qualify** — this diagram exists so you can recognise the minority that does, and so you are not talked into it by a picture.

Two separate choices that people constantly merge: **how events travel**, and **where they come to rest**.

The nine steps

- 1 · **Producers emit events.** Tills, applications, sensors. They fire and forget; nothing downstream is allowed to block them.
- 2 · **Choose a transport.** **Kinesis Data Streams** gives durable ordered shards with several independent consumers. **Amazon MSK** (or Confluent Cloud) is the right call when the organisation already speaks Kafka — the tooling and the people are already there. Ordering is guaranteed *within* a shard or partition, not across them, which is why the partition key matters more than it looks.
- 3 · **Path A — Firehose.** Managed delivery with no code at all. It buffers, batches and writes for you.
- 4 · **Firehose lands in Iceberg.** Including **S3 Tables**, with insert / update / delete **routed from a single stream** into different tables. The data is now in the lake, governed by the catalog, readable by every engine.
- 5 · **The trade-off nobody mentions.** ★ Firehose trades **ingest throughput against the number of Iceberg partitions open at once**. A stream fanning across many partitions delivers less throughput than one writing to a few. This is a constraint on your *partitioning design*, not a tuning knob you turn later.
- 6 · **Path B — straight into Redshift.** Same transport, different landing.

7 · **A materialized view mapped onto the stream.** ★ Redshift streaming ingestion maps an MV directly onto Kinesis or Kafka, refreshed from the stream **with no S3 staging at all**. Lower latency and one fewer moving part — and it is the same object from Module O L12, doing a new job.

8 · **The dashboard is seconds behind.** Which is genuinely impressive, and is the point at which you should ask the next question.

9 · **Ask it before you build.** ★ **Who is watching the screen at 3am?** And: **what decision changes in the next sixty seconds?** If the honest answers are "nobody" and "none", you wanted a shorter batch interval, not a stream — and you have just avoided an always-on cost that runs whether anyone looks or not.

Path A or Path B

	A · FIREHOSE → LAKE	B · STRAIGHT TO REDSHIFT
Latency	seconds to a minute	seconds
Lands in	Iceberg, governed, reusable	one materialized view
Read by	every engine	Redshift only
Keeps evidence	yes — raw events retained	no
Best for	events you will reprocess	one live dashboard

Path A keeps the evidence and feeds everything. **Path B is faster and feeds one thing.** Most projects want A and think they want B.

When you need more than delivery

If the requirement involves **windows**, **joins between streams**, or **remembering what you saw**, neither path is enough — that is **Managed Service for Apache Flink** or **Glue streaming ETL**. Delivery services buffer; they do not compute.

D24 · Streaming — Two Landing Paths

On Apparel Group

Neither platform streams today, and nothing in the SOW requires it. Learn it to recognise the case that does, and to price it honestly when someone asks.

Batch until it genuinely hurts. Then measure what hurts, and stream only that.

Checklist

- ☐ I can separate transport from landing
- ☐ I know Firehose writes to Iceberg including S3 Tables

- ☐ I know the throughput-versus-open-partitions trade
- ☐ I know streaming ingestion lands on an MV with no S3 hop
- ☐ I ask who is watching, and what decision changes, before agreeing to build
- ☐ I reach for Flink only when windows or state are actually required

You've got it when you can...

...hear "we need real-time" in a requirements session, ask what decision changes in the next minute, and land on the honest answer — usually an hourly batch at a fraction of the cost.

Orchestration And The Barrier

Ten steps. Step five is one conditional write, and it is the only thing standing between you and two dispatchers doing the same work.



IN PLAIN ENGLISH

Unload every lorry before anyone stacks a shelf. The barrier is the foreman calling “all in” — and only one foreman is allowed to call it.

D25 · Orchestration And The Barrier

Diagram: [_render/D25-orchestration-barriers.html](#)

What this pattern is for

Running many sources in parallel, letting each fail on its own, and then **starting the next phase only when every branch of the previous one has genuinely finished**.

The hard part is not the parallelism. It is the gate — and the gate is one conditional write.

The ten steps

1 · One schedule, one entry point. EventBridge Scheduler fires once. Not one cron per source — that is how you end up with a schedule nobody can reason about and two jobs racing at 02:00.

2 · A dispatcher reads the specs. It works out **which sources are due tonight** from configuration and control-plane state, not from a hardcoded list. This is what makes adding a source a config change rather than a deployment.

3 · A Map state fans out. One branch per source, running in parallel. Step Functions owns the concurrency limit, so you cap it in one place rather than hoping.

4 · Each branch fails independently. ★ Source B's slow API cannot mark source A as failed. Separate executions, separate retries, separate alarms. Without this, one bad feed marks the whole night red and nobody can tell which seven succeeded.

5 · The barrier. ★ Every branch writes its terminal state to DynamoDB with a **conditional write** — `attribute_not_exists(pk)` . The first writer wins; a second attempt fails harmlessly. This is the entire mechanism behind single-flight locks, and it is why two dispatchers firing at once cannot both proceed.

6 · The phase gate asks one question. ★ *Are all phase-1 branches in a terminal state?* Not "did they succeed" — **terminal**, which includes failed. A failed source must not hold the gate closed forever; it must be visible and skipped, or the whole platform stalls behind one broken feed.

7 · Phase 2 fans out the same way. Same shape, same isolation. Bronze MERGEs on the natural key.

8 · Everything is re-runnable. Because phase 2 merges rather than appends, re-running a branch is safe — which is what turns an incident into a retry.

9 · Backfill is the same path. ★ Not a separate script. The same state machine, given a date range. A backfill mechanism that differs from the nightly path is a mechanism nobody trusts at 6am.

10 · Alarm on branches that never end. A branch stuck in "running" is invisible to a failure alarm. Alarm on **duration and on missing terminal state**, not only on error.

The condition, in full

```
ConditionExpression: attribute_not_exists(pk)
```

That single expression gives you: single-flight locks, idempotent phase transitions, and safety against two schedulers. It is worth understanding properly, because everything above depends on it.

D25 · Orchestration And The Barrier

What breaks if you skip a piece

- **Cron per source** — races, and a schedule nobody can reason about.
- **No isolation** — one slow API fails the night; the other seven are unknown.
- **Gate on success, not terminal state** — one broken source blocks the platform indefinitely.
- **A separate backfill script** — used rarely, tested never, wrong when it matters.
- **Alarm only on failure** — a hung branch is silent.

On Apparel Group

Eight sources means eight branches from day one. Build the Map/barrier shape with **two** sources and prove the gate behaves when one of them fails on purpose — at two sources, not at eight.

Checklist

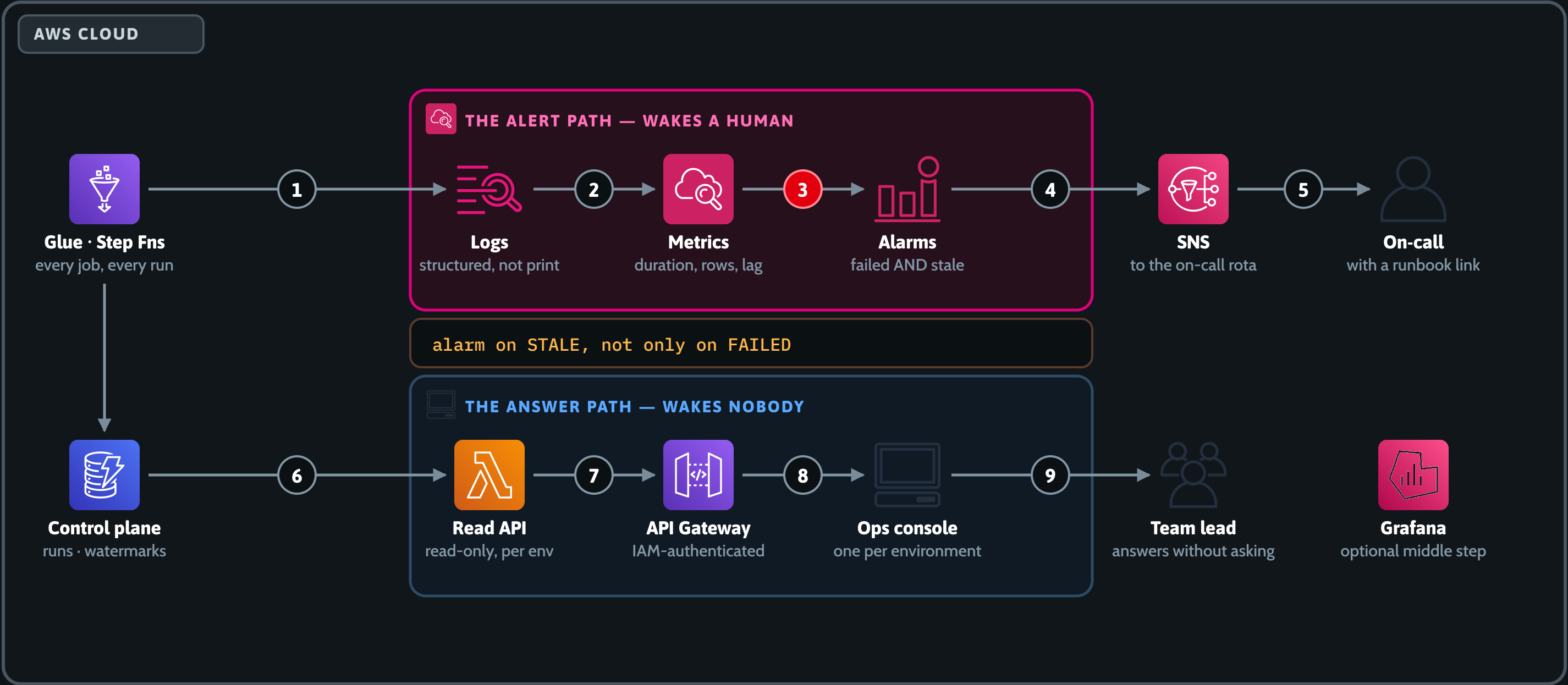
- ☐ One schedule, one entry point
- ☐ The dispatcher reads config, not a hardcoded list
- ☐ Branches fail independently, with their own retries and alarms
- ☐ Barrier uses a conditional write
- ☐ The gate waits for **terminal**, not for success
- ☐ Backfill uses the same state machine, with a date range
- ☐ Alarms cover duration and missing terminal state

You've got it when you can...

...deliberately fail one source in dev and show that the other seven complete, the gate opens, and exactly one alarm fires — naming the row in DynamoDB that made it safe.

Observability And The Ops Console

Two paths out of every job: one wakes somebody up, the other answers “did last night work?” without waking anybody.



IN PLAIN ENGLISH

A fire alarm and a noticeboard. Most questions are noticeboard questions — and a platform with only an alarm makes everyone ask a person.

D26 · Observability And The Ops Console

Diagram: [_render/D26-observability.html](#)

What this pattern is for

Two completely different questions, which need two completely different paths:

- "Something is broken, wake someone."
- "Did last night work?" — asked at 9am, by someone who does not want to open CloudWatch.

A platform with only the first path forces every routine question through a person. That person becomes the observability layer, and they get tired.

The nine steps

1 · **Jobs emit structured logs.** Not `print()` . Structured events with the run id, source, table and row counts, so they can be queried rather than read.

2 · **Metrics come out alongside.** Duration, rows processed, watermark lag. Metrics are what you alarm on; logs are what you read afterwards to find out why.

3 · **Alarm on stale, not only on failed.** ★ The dangerous state is **silence**. A job that failed loudly gets fixed by lunchtime; a job that stopped being scheduled three weeks ago and never errored is the one that puts a wrong number in a board pack. Alarm on **freshness** — "this table has not been updated in N hours" — as well as on error.

4 · **SNS routes to the on-call rota.** Not to a shared mailbox nobody owns. An alert with no named recipient is a log line with extra steps.

5 · **The alert carries a runbook link.** ★ An alarm that says "GlueJobFailed" and nothing else costs the responder twenty minutes of orientation. One line — *what this means, what to check first, how to re-run safely* — turns a

page into a task.

6 · **The answer path starts at the control plane.** Run state already lives in DynamoDB (D08): what ran, when, how long, with what outcome. Nothing extra needs to be collected.

7 · **A read-only API in front of it.** Small, read-only, deployed per environment. It cannot start, stop or retry anything — that constraint is what makes it safe to give out.

8 · **The ops console reads it.** ★ **One console per environment**, not a global one with an environment switcher. Prod has its own, with its own access. The switcher is how someone reads dev numbers in a prod incident.

9 · **The team lead answers without asking anyone.** Which is the actual measure of success. If "did XStore load?" still requires a message to an engineer, the console has not landed regardless of how good it looks.

Optionally in the middle

Amazon Managed Grafana is a reasonable middle step: better than raw CloudWatch dashboards, far cheaper than building a console. The two-tier plan is Grafana first, custom console when the audience is non-technical.

The two-path rule

	ALERT PATH	ANSWER PATH
Trigger	something broke	someone asked
Latency	seconds	whenever
Audience	on-call engineer	team lead, analyst, client
Ends in	a page with a runbook	a screen with a number
Cost of missing it	an incident	everyone messages an engineer

D26 · Observability And The Ops Console

What breaks if you skip a piece

- No freshness alarm — silent staleness, discovered in a report.
- No runbook link — every page starts from zero.
- No answer path — routine questions become interruptions.
- A global console — someone reads the wrong environment during an incident.

On Apparel Group

Ship the console **with** the platform, not after it. It reads the control plane, which exists from day one anyway, so the marginal cost is small — and it is what makes handover (MO5) possible at all.

Checklist

- ☐ Logs are structured and carry the run id

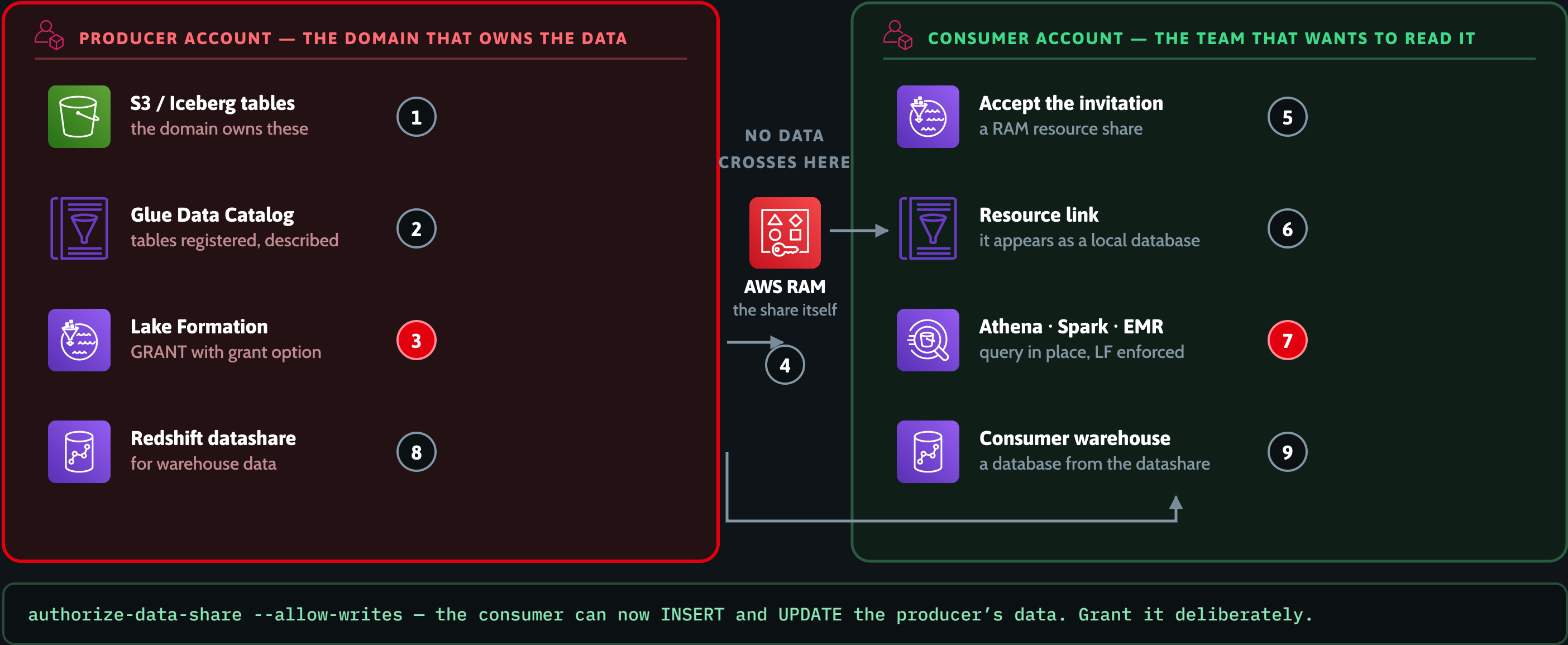
- ☐ Metrics include duration, rows and lag
- ☐ There is a **freshness** alarm, not only a failure alarm
- ☐ Alerts route to a named rota
- ☐ Every alarm has a runbook line
- ☐ Run state is queryable, not only logged
- ☐ The console is read-only and per-environment

You've got it when you can...

...point at a table that has been silently stale for three days and show the alarm that would have caught it — then explain why nobody noticed without it.

Sharing Across Accounts

How a mesh is actually plumbed. Nine steps, two accounts, and not one byte of data copied between them.



IN PLAIN ENGLISH

You are handed a key to their filing room, not a photocopy of the file. They can change the lock at any time, and you will feel it immediately.

D27 · Sharing Across Accounts

Diagram: [_render/D27-cross-account-sharing.html](#)

What this pattern is for

Letting another team read your data **without giving them a copy of it**. This is how a data mesh (Module 0 LO5) is actually plumbed on AWS — the organisational parts are yours to build, but the sharing is well supported.

The thing to notice on the diagram: **no data crosses the middle**. What crosses is permission.

The nine steps

- 1 · **The producer owns the storage.** S3 and Iceberg tables in the producing team's account. They stay there. Ownership is not transferred, delegated or duplicated.
- 2 · **The tables are registered in the catalog.** Named, described, discoverable. A dataset nobody can find is not shared however permissive the policy is.
- 3 · **Lake Formation grants, with the grant option.** ★ The producer grants at **catalog, database, table or column** level. Two styles: **named-resource** grants (explicit, one per resource) and **tag-based, LF-TBAC** (grant by classification — this is the one that scales past a handful of tables).
- 4 · **AWS RAM carries the share.** ★ RAM shares the *resource*, not the bytes. This is the step people expect to be a copy job and it is not one — nothing is transferred, staged or synchronised.
- 5 · **The consumer accepts the invitation.** An explicit action in the consuming account. Sharing is not something that happens *to* an account; someone there has to accept it, which is what makes the audit trail meaningful.
- 6 · **A resource link makes it local.** ★ The shared database appears in the consumer's own catalog under a name they choose. From this point their tooling treats it like any other database — no special paths, no cross-account syntax in every query.

7 · **Query in place, still governed.** ★ Athena, Spark and EMR read it directly, and **Lake Formation still enforces the producer's grants**. Column masking applied by the producer is applied here too. Governance does not stop at the account boundary — this is the property that makes the whole pattern trustworthy.

8 · **Warehouse data uses a datashare instead.** For Redshift-to-Redshift, the mechanism is a **datashare**: a producer/consumer container sharing live data across clusters, accounts and Regions.

9 · **The consumer creates a database from it.** ★ And — the correction most material misses — **datashares are no longer read-only**:

```
aws redshift authorize-data-share \  
  --data-share-arn arn:aws:redshift:...:datashare:../salesshare \  
  --consumer-identifier <consumer> \  
  --allow-writes
```

With `--allow-writes` the consumer can `INSERT` and `UPDATE` the producer's data. That is a serious grant. Give it deliberately, and know that most documentation written before it landed says it is impossible.

Why this beats sending extracts

	SHARED IN PLACE	A NIGHTLY EXTRACT
Freshness	live	as of last night
Governance	producer's grants still apply	gone the moment it lands
Cost	consumer's own compute	storage twice, plus a pipeline
Revocation	immediate	you cannot un-send a file
Drift	impossible	inevitable

D27 · Sharing Across Accounts

That fourth row is the one to lead with in a governance conversation.

What breaks if you skip a piece

- **Named-resource grants at scale** — a grant per table per consumer, and nobody can audit it. Use tags.
- **No resource link** — consumers write awkward cross-account references everywhere.
- **Assuming governance stops at the boundary** — it does not, and that is the point; teams who assume otherwise build a second masking layer for nothing.
- `--allow-writes` **granted casually** — cross-account write access to your data.

On Apparel Group

Not needed at go-live: one platform team, one account per environment. It becomes relevant if Apparel Group later wants a brand or a region to own and publish its own data — and it is worth knowing now, because the answer to *"can another team have this data?"* should be **"yes, shared"**, never **"yes, I'll send you a file."**

Checklist

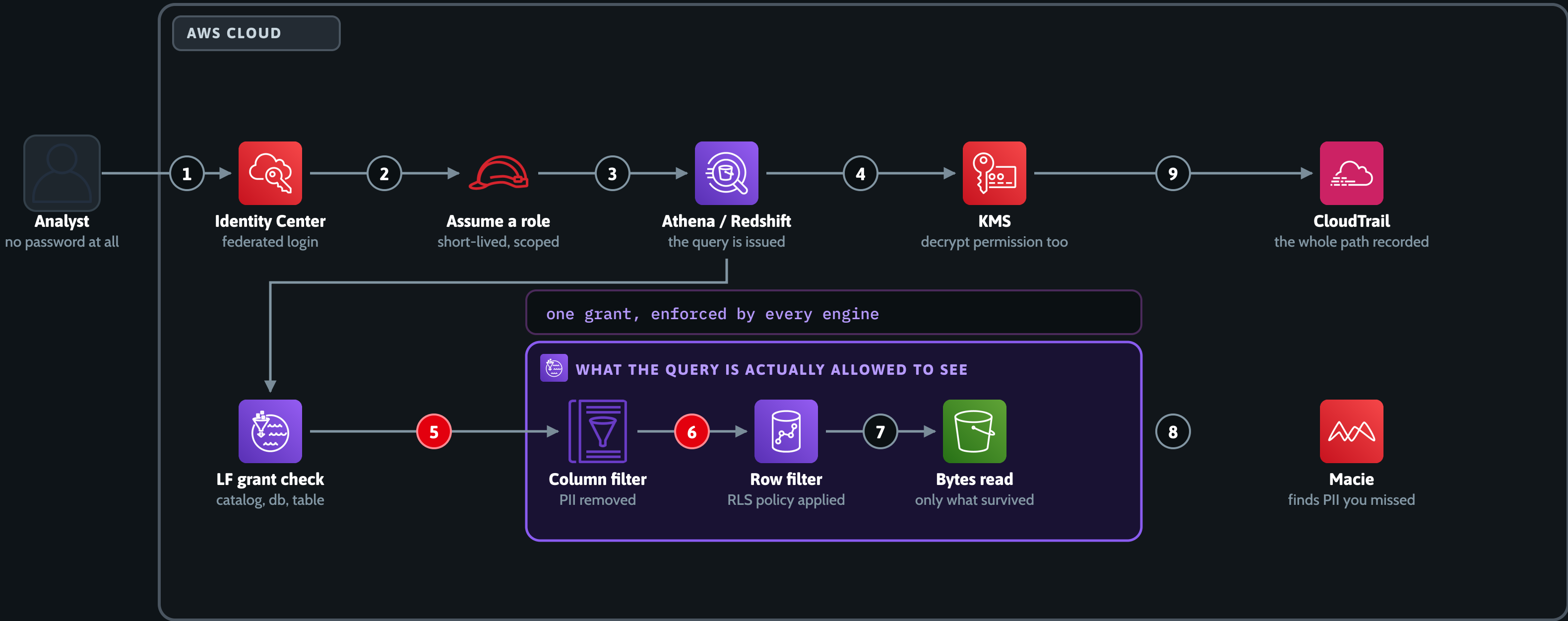
- ☐ Producer retains ownership of storage
- ☐ Grants are tag-based where there is more than a handful of tables
- ☐ Consumer explicitly accepts the RAM share
- ☐ A resource link exists so it looks local
- ☐ Column-level masking is verified **from the consumer side**
- ☐ `--allow-writes` is granted only where genuinely intended
- ☐ Nobody is emailing extracts

You've got it when you can...

...be asked for "a copy of the sales data for the marketing team" and set up a governed share instead — then show that a masked column is still masked when they query it.

One Query, Every Control It Passes

Follow a single analyst's SELECT through nine checks. If any one of them is missing, the others do not save you.



IN PLAIN ENGLISH

Badge at the door, key to the room, and a cover over the parts of the file you may not read. Every step logged, so it can be proved afterwards.

D28 · One Query, Every Control It Passes

Diagram: [_render/D28-security-deep-dive.html](#)

What this pattern is for

Security on a lakehouse is not one gate. It is nine, and they are owned by different services. Following a single analyst's `SELECT` through all of them is the fastest way to understand which one you are actually debugging.

If any one control is missing, the others do not save you.

The nine checks

1 · **The analyst has no database password.** Not "a strong password" — none. Removing the credential removes the whole class of problems that come with rotating, sharing and leaking one.

2 · **Identity Center authenticates them.** Federated login against the corporate directory. Joiners and leavers are handled where they are already handled, not in a second place someone has to remember.

3 · **They assume a role.** Short-lived, scoped credentials. The role — not the person — carries the permissions, which is why a leaver is removed from a group rather than hunted through scattered grants.

4 · **The query is issued.** Athena or Redshift. Note that nothing so far has considered *the data* — everything up to here is about identity.

5 · **Lake Formation checks the grant.** ★ At catalog, database, table **and column** level. This is where authorisation actually happens for anything read through the catalog, and — crucially — **the same grant is enforced by every engine**. Athena, Spark, EMR and Redshift Spectrum all honour it. Define once, applies everywhere.

6 · **Columns are filtered out.** ★ PII columns the role was not granted simply **do not appear in the result**. Not blanked, not masked in the report — absent from what the engine returns. This is the difference between a control and a convention.

7 · **Rows are filtered.** Row-level security in Redshift narrows what the role can see: one region's stores, one brand's products. Applied by the database, not by whoever wrote the dashboard.

8 · **Only the surviving bytes are read.** Which is a nice property: the column and row filters reduce cost as well as exposure. Governance and the bill point the same way for once.

9 · **KMS and CloudTrail bracket the whole thing.** ★ The role also needs **decrypt permission on the KMS key** — a grant on the table is not enough if the key says no, which is the single most confusing "access denied" on a lakehouse. And **CloudTrail records the entire path**, so the question "who read the customer table last March?" has an answer.

Macie runs alongside, scanning for PII in places nobody classified. It is how you find the column somebody added six months ago and never told anyone about.

Where the "it works for me" bugs come from

SYMPTOM	USUALLY
Works in Athena, not in Redshift	two grant systems — Redshift <code>GRANT</code> vs Lake Formation (Module O L17)
Access denied with a valid table grant	KMS key policy , not the table grant
Sees a column they should not	grant made at table level instead of column level
Sees all rows	RLS policy exists but the role is exempt
Nobody can explain who granted it	it was clicked in the console, not written in Terraform

D28 · One Query, Every Control It Passes

That last row is why every grant belongs in Terraform: an access grant made in the console has no author, no reason and no reviewer — and nobody will ever dare remove it.

On Apparel Group

Epsilon carries customer PII. The control that matters is step 6: masked **at the column**, never by maintaining a separate filtered copy. A copy is a second thing to secure, a second thing to keep in sync, and a second thing to leak.

Verify masking **from the consumer's seat**, not from an admin session. An admin can see everything, which makes admin a useless place to test from.

Checklist

- ☐ No database passwords anywhere

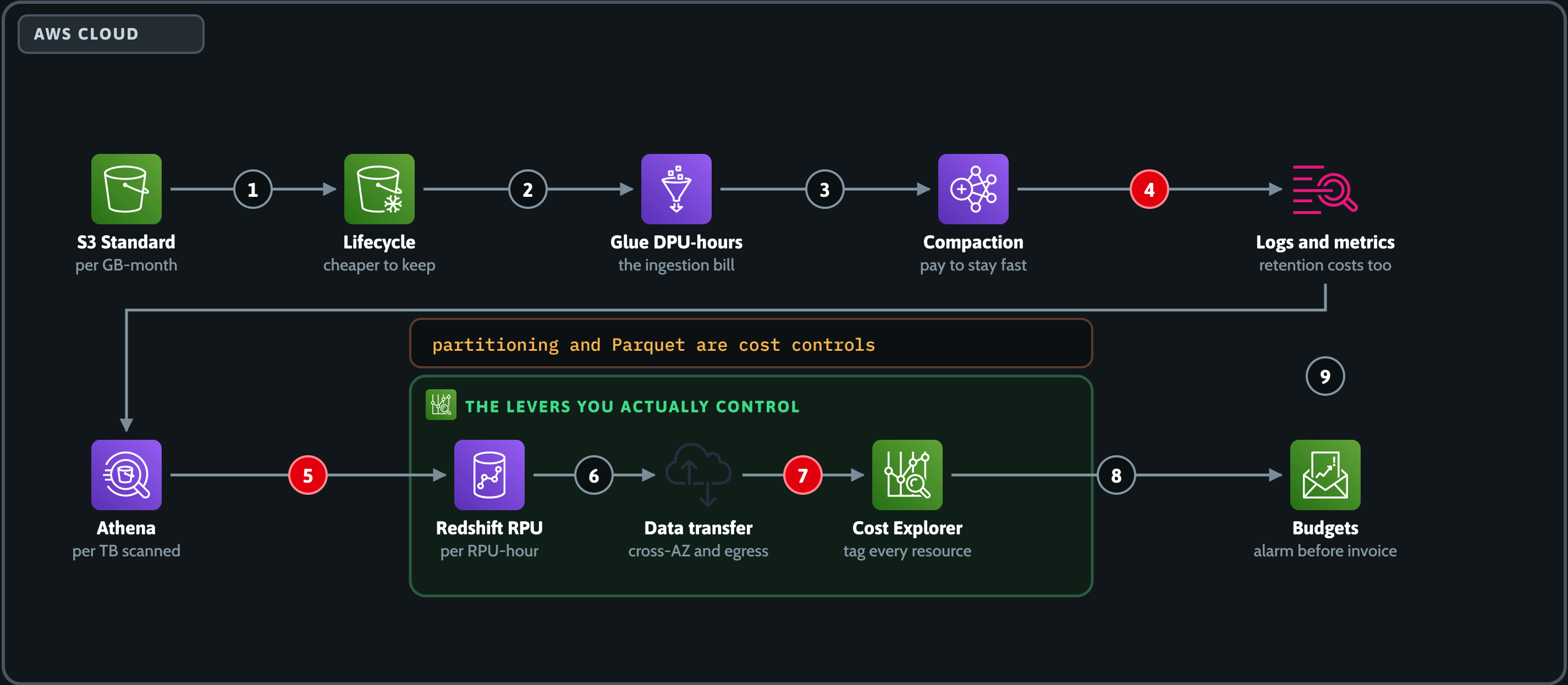
- ☐ Grants go to roles, never to individuals
- ☐ PII is column-level granted, not report-filtered
- ☐ RLS applied where a role should see only some rows
- ☐ The role has **KMS decrypt** as well as the table grant
- ☐ CloudTrail is on and retained long enough to answer an audit
- ☐ Every grant lives in Terraform
- ☐ Masking tested from a real consumer role

You've got it when you can...

...be handed "access denied" on a table the user was definitely granted, and check the KMS key policy before anything else — because that is the one nobody thinks of.

Following The Money

Nine places a lakehouse spends. Only three of them appear on the diagram people usually draw.



IN PLAIN ENGLISH

Storage is the cheap part. What costs money is reading it badly, over and over, on a schedule nobody has reviewed since the day it was set.

D29 · Following The Money

Diagram: [_render/D29-cost-and-lifecycle.html](#)

What this pattern is for

Nine places a lakehouse spends money. **Only three of them appear on the diagram people usually draw** — and the six that do not are where the surprises live.

The nine places

- 1 · **S3 Standard, per GB-month.** The cheap part, and the one everybody worries about. Storage is rarely the problem.
- 2 · **Lifecycle transitions.** Moving cold partitions to IA, then Glacier Instant Retrieval, cuts the storage line. ⚠️ But **retrieval is billed separately** — archiving data a dashboard still reads is a cost *increase* that hides as a saving, because the saving lands on one line and the cost on another (Module O L27).
- 3 · **Glue DPU-hours.** The ingestion bill. Driven by how much you read, how parallel it is, and how often it runs. Halving a schedule halves this line, and most schedules were set once and never revisited.
- 4 · **Compaction.** ⭐ The line nobody budgets for. Iceberg v2 writes delete files rather than rewriting data; without scheduled compaction, reads get slower and storage grows for data that is logically gone. **You are paying either way** — either compaction DPU-hours, or slower queries forever.
- 5 · **Logs and metrics retention.** CloudWatch keeps what you tell it to keep, and the default is often "forever". On a chatty platform this becomes a real number quietly.
- 6 · **Athena, per TB scanned.** ⭐ The most controllable line on the whole list. Partitioning and Parquet are **cost controls**, not performance tuning — a query against unpartitioned CSV can cost a hundred times the same

query against partitioned Parquet. Set a per-query scan limit on the workgroup so a bad query is an error, not an invoice.

- 7 · **Redshift RPU-hours.** Serverless bills for what you use, which is excellent right up until an always-on dashboard keeps it warm all night. Check what is querying it outside business hours.
- 8 · **Data transfer.** Cross-AZ traffic and egress. Usually small, occasionally startling — a cross-Region read in a loop is the classic. Worth checking once rather than assuming.
- 9 · **Cost Explorer and Budgets.** ⭐ **Tag every resource by environment and by pipeline from day one.** Untagged spend cannot be attributed, and unattributed spend cannot be reduced — you end up arguing about a total instead of fixing a line. Set a budget alarm so you find out before the invoice does.

The levers you actually control

LEVER	AFFECTS	TYPICAL EFFECT
Partitioning	Athena, Spectrum, Glue	the largest single lever
Parquet instead of CSV/JSON	everything that reads	10–100× less scanned
File size (128 MB–1 GB)	Glue, Athena	fewer, bigger reads
Schedule frequency	Glue DPU-hours	linear
Workgroup scan limits	Athena	caps the worst case
Lifecycle rules	S3 storage	check what still reads it first
Log retention	CloudWatch	quiet but real

D29 · Following The Money

The three questions for a cost review

- 1. What is the biggest line, and what drives it? Not "what is expensive" — what *causes* it.
- 2. What runs on a schedule nobody has reviewed? Schedules are set once, in a hurry, and never revisited.
- 3. What is untagged? If you cannot attribute it, you cannot reduce it.

On Apparel Group

XStore is the giant, and it will dominate lines 3, 4 and 6. Two decisions to make deliberately before go-live rather than after the first invoice:

- Its **partitioning scheme** — it sets the floor for every query cost afterwards, and it is the hardest thing to change later.
- Its **ingestion cadence** — this is also the cost argument in the zero-ETL comparison (D17). Continuous replication of a very large, high-churn table has an ongoing cost that a nightly batch does not.

Checklist

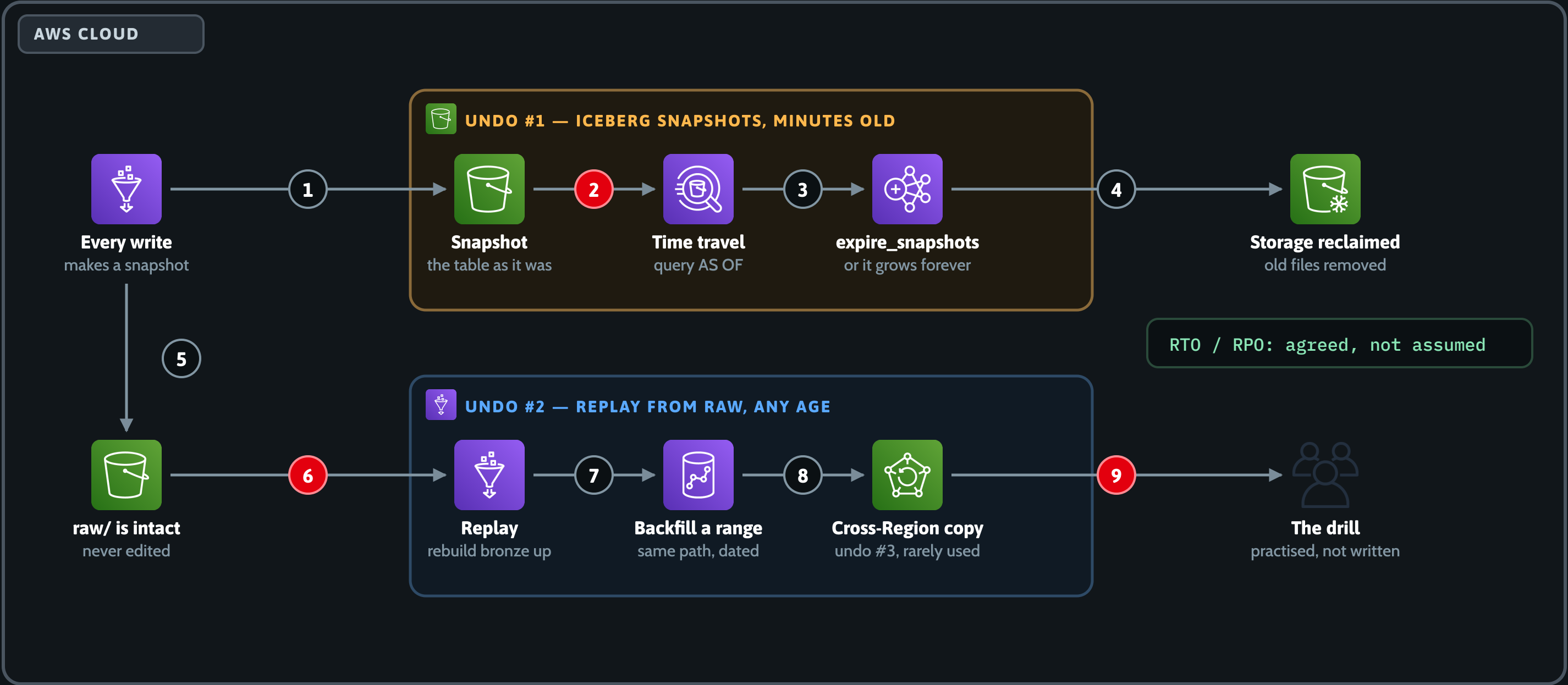
- ☐ Every resource tagged by environment and pipeline
- ☐ A budget alarm exists
- ☐ Athena workgroups have a per-query scan limit
- ☐ Largest tables are partitioned on what people filter by
- ☐ Everything queried is Parquet, not CSV or JSON
- ☐ Compaction is scheduled and its cost is known
- ☐ Log retention is set deliberately
- ☐ Lifecycle rules checked against what still reads the data

You've got it when you can...

...be shown a rising bill and name the three lines to look at first — then find the schedule somebody set in week two and nobody has looked at since.

Recovery, Backfill And Time Travel

Three different “undo” buttons for three different disasters. Knowing which one you need is the skill.



IN PLAIN ENGLISH

Snapshots undo this morning. Raw undoes any morning. A backup nobody has restored from is a folder, not a recovery plan.

D30 · Recovery, Backfill And Time Travel

Diagram: [_render/D30-dr-backfill-time-travel.html](#)

What this pattern is for

Three different "undo" buttons for three different disasters. Knowing which one you need — and how far back it reaches — is the skill. Reaching for the wrong one under pressure is how a bad morning becomes a bad week.

UNDO	REACHES BACK	USE IT FOR
Iceberg snapshots	minutes to days	a bad load this morning
Replay from raw	any age	a business rule that was wrong for months
Cross-Region copy	last replication	the Region is gone

The nine steps

- 1 · Every write creates a snapshot. Not a backup job you scheduled — a property of the table format. Iceberg records the exact set of files that constituted the table at that commit.
- 2 · The snapshot is the table as it was. Immutable, cheap (it references existing files rather than copying them), and available immediately.
- 3 · Time travel queries it. ★ `SELECT ... FOR TIMESTAMP AS OF ...` — the table as of this morning, before the bad load. This is how you *diagnose* before you decide, rather than restoring blind and hoping.
- 4 · `expire_snapshots` must run. ★ Snapshots retain the files they reference. Without expiry, storage grows forever and metadata gets slower. **Choose the retention window deliberately** — it is exactly how far back your fastest undo reaches, so "7 days" is a recovery decision, not a housekeeping one.

5 · **Storage is reclaimed.** Files no snapshot references are deleted. This is the moment your fastest undo stops reaching that far back — worth saying out loud when someone proposes shortening the window to save money.

6 · `raw/` is intact, always. ★ Written once, never edited, never expired within the retention period. This is the reason the medallion architecture insists raw is immutable: it is the second undo button, and it reaches back as far as your retention.

7 · **Replay rebuilds upward.** Bronze and silver are *derived*. If a business rule was wrong for six months, you fix the rule and reprocess from raw. Nothing was lost because nothing downstream was the only copy.

8 · **Backfill is a date range on the normal path.** ★ Not a separate script. The same job, the same MERGE, given different dates — which works precisely because loads are idempotent (Module O L11). A backfill mechanism that differs from the nightly path is one nobody trusts at 6am.

9 · **The drill is practised, not written.** ★ **A backup nobody has restored from is a folder, not a recovery plan.** Run the drill: restore a table from a snapshot, replay a day from raw, and time both. Those two numbers are your real RTO — everything else is an assumption.

RTO and RPO — agreed, not assumed

- **RPO** (how much data you can afford to lose) is set by ingestion cadence. A nightly batch has an RPO of one day. Saying so plainly at design time avoids a difficult conversation later.
- **RTO** (how fast you can be back) is what the drill measures.

Both belong in writing, agreed with the business, before go-live. They are the two numbers everyone assumes are better than they are.

D30 · Recovery, Backfill And Time Travel

What breaks if you skip a piece

- No `expire_snapshots` — storage grows forever, metadata slows, cost rises with no obvious cause.
- Expiry too aggressive — your fast undo no longer reaches the incident.
- Raw not immutable — the second undo button does not exist, and you did not find out until you needed it.
- A separate backfill script — rarely used, never tested, wrong when it matters.
- No drill — RTO is a guess, and it is optimistic.

On Apparel Group

Set the snapshot retention window in **week one**, alongside the S3 lifecycle policy (D29) — the two interact, and both are painful to change once there is history.

Run the recovery drill **before go-live**, in QA, with the client team watching. It is also the most convincing single thing you can show at handover (MO5): not that the platform works, but that it can be *fixed*.

Checklist

- ☐ Snapshot retention window chosen deliberately and written down
- ☐ `expire_snapshots` scheduled and monitored
- ☐ `raw/` is immutable and its retention exceeds the replay window
- ☐ Backfill uses the normal path with a date range
- ☐ Cross-Region copy exists if the RPO requires it
- ☐ RTO and RPO agreed with the business, in writing
- ☐ The drill has been **run**, and both timings recorded

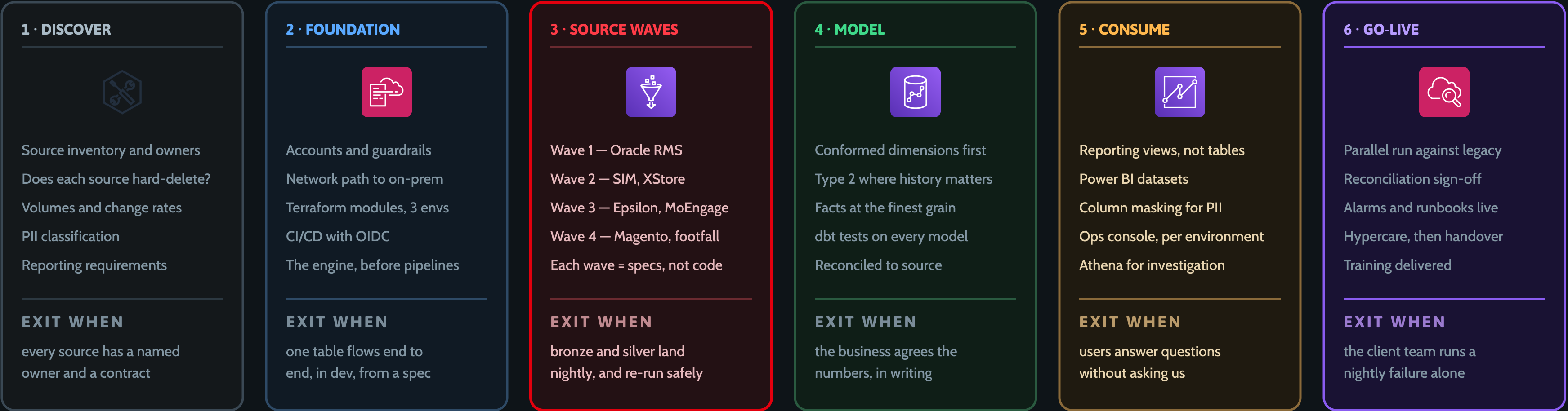
You've got it when you can...

...be told at 8am that yesterday's load wrote bad data, and choose between time travel and a replay in under a minute — because you already know how far back each one reaches.

The Programme, End To End

Six phases from kickoff to hypercare. Everything in the other diagrams sits inside one of these boxes.

1 · THE DELIVERY SEQUENCE



THE THINGS THAT DELAY PROGRAMMES LIKE THIS — START THEM IN PHASE 1

Network access to on-prem Oracle (weeks of lead time) · PII classification and legal sign-off · source system owners agreeing to a contract · a decision on the Oracle ingestion route

IN PLAIN ENGLISH

Six phases, and the only ones that slip are the ones waiting on somebody else. Start those on day one, even if you cannot finish them yet.

M01 · The Programme, End To End

Diagram: [_render/M01-programme-master-flow.html](#)

What it shows

The whole engagement in six phases. **Everything in the other 34 diagrams sits inside one of these boxes.**

This is a sponsor's diagram, not an engineer's. It answers "where are we, and what happens next" without anyone needing to know what Iceberg is.

The six phases, and what closes each

PHASE	EXIT WHEN
1 · Discover	every source has a named owner and a contract
2 · Foundation	one table flows end to end, in dev, from a spec
3 · Source waves	bronze and silver land nightly, and re-run safely
4 · Model	the business agrees the numbers, in writing
5 · Consume	users answer questions without asking us
6 · Go-live	the client team runs a nightly failure alone

Each exit criterion is deliberately something you can **demonstrate**, not something you can assert. "Foundation is done" is an opinion; "one table flows end to end from a spec" is a demo.

Phase 1 carries the questions that matter

Discovery is not a formality. Three things are established here or they cost you later:

- **Does each source hard-delete?** Decides the ingestion mechanism per source (D17, D22).
- **What is PII, and who signs off?** Blocks wave 3 entirely (M04).
- **Volumes and change rates.** Decides parallelism, cost and whether zero-ETL is even affordable.

The things that actually delay programmes like this

None of them are engineering:

1. **Network access to on-prem Oracle** — weeks of lead time, and not ours to expedite (D18)
2. **PII classification and legal sign-off** — blocks the Epsilon wave
3. **Source system owners agreeing a contract** — blocks everything
4. **The Oracle ingestion decision** — needs a measured spike, not a meeting

Start all four in phase 1, even though none of them can finish there. A dependency you cannot complete is still a dependency you can begin.

Phase 6 is a capability, not a date

"Go-live" is often written as a date and treated as a handover. It is neither. The exit criterion is that **the client team can handle a nightly failure without us** — which means runbooks written, alarms tuned, the ops console in use, and at least one real incident handled by them while we watch.

Anything less and hypercare never ends; it just stops being funded.

M01 · The Programme, End To End

Checklist

- ☐ I can name the six phases and each exit criterion
- ☐ Every exit criterion is demonstrable, not assertable
- ☐ The four blockers were started in phase 1
- ☐ Phase 4 ends with written agreement on the numbers

- ☐ Phase 6 ends with a capability, not a date

You've got it when you can...

...present this to a sponsor and answer "when will it be done?" by pointing at the exit criteria — and by naming which of the four blockers is currently the critical path.

Source To Dashboard, Traced

“Where does this number come from?” — answered for all eight sources, in one row each.

SOURCE	LANDS AS	SILVER ENTITY	GOLD MART	ANSWERS
ORACLE Oracle RMS	bronze.rms_*	product, supplier, price	dim_product	margin, assortment
ORACLE Oracle SIM	bronze.sim_*	stock position, movement	fct_stock_daily	availability, shrinkage
ORACLE Oracle XStore	bronze.xstore_*	receipt, sale line, tender	fct_sales_line	sales, basket, VAT
 Epsilon (PII)	bronze.epsilon_*	customer, loyalty member	dim_customer	loyalty, retention
 MoEngage	bronze.moengage_*	campaign, engagement	fct_campaign	campaign response
 Magento	bronze.magento_*	online order, web customer	fct_sales_line	online vs store
 Vemco footfall	bronze.vemco_*	store visit count	fct_footfall	conversion rate
 Irisys footfall	bronze.irisys_*	store visit count	fct_footfall	conversion rate
dim_store and dim_date are conformed — every fact above joins to the same two, or the marts will disagree				

IN PLAIN ENGLISH

One row per source, left to right. When a figure is questioned, you follow its row back to the system that produced it — and the argument ends.

M02 · Source To Dashboard, Traced

Diagram: [_render/M02-source-to-dashboard.html](#)

What it shows

"Where does this number come from?" — answered for all eight sources, one row each, left to right:



The eight rows

SOURCE	LANDS AS	SILVER ENTITY	GOLD MART	ANSWERS
Oracle RMS	bronze.rms_*	product, supplier, price	dim_product	margin, assortment
Oracle SIM	bronze.sim_*	stock position, movement	fct_stock_daily	availability, shrinkage
Oracle XStore	bronze.xstore_*	receipt, sale line, tender	fct_sales_line	sales, basket, VAT
Epsilon (PII)	bronze.epsilon_*	customer, loyalty member	dim_customer	loyalty, retention
MoEngage	bronze.moengage_*	campaign, engagement	fct_campaign	campaign response
Magento	bronze.magento_*	online order, web customer	fct_sales_line	online vs store
Vemco footfall	bronze.vemco_*	store visit count	fct_footfall	conversion rate
Irisys footfall	bronze.irisys_*	store visit count	fct_footfall	conversion rate

Two things to notice

Two sources land in one fact. XStore and Magento both feed `fct_sales_line` — that is deliberate, and it is how "online vs store" becomes a filter rather than a separate report. Same for the two footfall vendors feeding `fct_footfall` : two vendors, one grain, conformed.

`dim_store` and `dim_date` are conformed. Every fact above joins to the same two. Without that, two marts that both report "sales by region" will disagree structurally — and the disagreement will not look like a bug, it will look like an argument.

Why this diagram ends arguments

When a figure is questioned in a meeting, you follow its row back to the system that produced it. Three consequences:

1. The conversation becomes "is the source right?" rather than "is your pipeline right?" — a different and usually shorter conversation.
2. **Impact analysis is visual.** If RMS changes, look down its row.
3. **Ownership is visible.** Every number has a source system, and every source system has an owner (MO1, phase 1).

Keep it current

This is the diagram that rots fastest, because it changes every time a mart is added. **Update it in the same PR that adds the mart.** A traceability diagram that is six months stale is worse than none — people trust it and are wrong.

M02 · Source To Dashboard, Traced

Checklist

- ☐ I can trace any gold table back to its sources
- ☐ I know which two sources share `fct_sales_line` and why
- ☐ I know the two conformed dimensions and what they prevent
- ☐ I know which row carries PII

- ☐ The diagram is updated in the PR that changes the model

You've got it when you can...

...be challenged on a number in front of a client and walk its row backwards to the source system — turning a credibility question into a data question.

The Retail Domain Model

Four dimensions and four facts. Every dashboard the business asks for is a combination of these eight things.

dim_store One row per version of a store Fed by Oracle RMS Type 2 — stores move region Conformed: every fact joins it store_sk · valid_from · valid_to	dim_product One row per version of an SKU Fed by Oracle RMS Type 2 — hierarchy changes Brand, department, category product_sk · pack_size parsed	dim_customer One row per loyalty member Fed by Epsilon — this is PII Masked at the column, not copied Lake Formation grants apply customer_sk · column-level grants	dim_date One row per calendar day Generated, not sourced Fiscal calendar, week, season Built once, before anything else date_sk · one timezone, written down
fct_sales_line One row = one line on one receipt Fed by XStore and Magento Additive: quantity, net, VAT The biggest table by far store_sk · product_sk · date_sk	fct_stock_daily One row = one SKU, store, day Fed by Oracle SIM Semi-additive — never sum over time Snapshot, not a movement log on_hand · on_order · in_transit	fct_footfall One row = one store, one hour Fed by Vemco and Irisys Two vendors, one grain — conform Small, but arrives unreliably visits → conversion vs sales	fct_campaign One row = one engagement event Fed by MoEngage Joins customers to sales Attribution is a business rule sent · opened · clicked · converted

IN PLAIN ENGLISH

Four things you slice by, four things you count. Build the four dimensions first — every fact after that is quicker than the one before.

M03 · The Retail Domain Model

Diagram: [_render/M03-retail-domain-model.html](#)

What it shows

Four dimensions and four facts. Every dashboard the business asks for is a combination of these eight things.

That is worth saying out loud early in a programme, because the request list will look like forty reports. It is eight objects, sliced differently.

Four dimensions — what you slice by

	GRAIN	FED BY	NOTE
<code>dim_store</code>	one row per version of a store	Oracle RMS	Type 2 — stores move region
<code>dim_product</code>	one row per version of an SKU	Oracle RMS	Type 2 — hierarchy changes; pack size parsed
<code>dim_customer</code>	one row per loyalty member	Epsilon	PII — masked at the column
<code>dim_date</code>	one row per calendar day	generated	fiscal calendar, one timezone, written down

Four facts — what you count

	GRAIN	FED BY	NOTE
<code>fct_sales_line</code>	one line on one receipt	XStore + Magento	the biggest table by far
<code>fct_stock_daily</code>	one SKU, store, day	Oracle SIM	semi-additive — never sum over time
<code>fct_footfall</code>	one store, one hour	Vemco + Irisys	two vendors, one grain
<code>fct_campaign</code>	one engagement event	MoEngage	attribution is a business rule

Build the dimensions first

`dim_date` and `dim_store` before anything else, then `dim_product` . Every fact joins to them, so building them first means each subsequent fact is faster than the one before — and building them late means every fact gets reworked.

This is why **wave 1 is Oracle RMS** (MO4): it is the reference data everything else attaches to.

The two traps on this diagram

`fct_stock_daily` is **semi-additive**. Stock on hand of 100 on Monday and 100 on Tuesday is not 200. Summing it over time is the most common wrong number in retail analytics, and it looks completely plausible in a chart.

`dim_customer` is **PII**. It is the only object here that carries a legal obligation. Column-level grants, not a filtered copy (D19).

Attribution is a business rule, not a technical one

`fct_campaign` joins customers to sales, and *how* it attributes a sale to a campaign — last touch, first touch, a window — is a decision the business makes and signs. Implement whichever they choose; do not choose for them, and do not let it be decided implicitly by whoever writes the SQL.

M03 · The Retail Domain Model

Checklist

- ☐ I can name the four dimensions and four facts with their grains
- ☐ I know which dimensions are Type 2 and why
- ☐ I know `fct_stock_daily` is semi-additive
- ☐ I know which object carries PII

- ☐ Dimensions are built before facts
- ☐ Attribution rules are agreed with the business, in writing

You've got it when you can...

...be handed a list of forty requested reports and map them onto these eight objects — then say which two or three actually need new modelling.

Delivery Waves And What Blocks Them

Sources are onboarded in waves, hardest first. What actually delays a wave is almost never the engineering.

1 · FOUR WAVES, HARDEST FIRST

WAVE 1 · PROVE IT

ORACLE Oracle RMS

The reference data everything else joins to — product, store, supplier, price.

Builds dim_product and dim_store first, so every later wave has something to join to.

PROVES

the engine works, end to end

WAVE 2 · THE VOLUME

ORACLE SIM · XStore

The two big transactional sources. XStore is the giant and carries the receipt logic.

This is where parallelism, cost and the hard-delete question all get answered for real.

PROVES

it scales, and it reconciles

WAVE 3 · THE APIS



Epsilon · MoEngage

A different connector class: rate limits, cursor paging, no bulk export to fall back on.

Epsilon carries PII, so this wave cannot start until classification and sign-off are done.

PROVES

governance actually works

WAVE 4 · THE REST



Magento · footfall

Small volumes, third connector class, unreliable arrival.

Deliberately last: by now the landing conventions, alarms and idempotency are proven, so late and duplicate files are harmless.

PROVES

the ninth source is cheap

THESE BLOCK WAVES AND ARE NOT OURS TO FIX — TRACK THEM FROM DAY ONE

Network path to on-prem Oracle → blocks waves 1–2 · PII classification and legal sign-off → blocks wave 3 · SaaS API credentials and quota → blocks wave 3 · source owners agreeing a contract → blocks everything

IN PLAIN ENGLISH

Do the hardest source first, while there is still time to be wrong about it. The easy ones at the end are the reward, not the warm-up.

M04 · Delivery Waves And What Blocks Them

Diagram: [_render/M04-delivery-waves.html](#)

What it shows

Sources onboarded in four waves, **hardest first** — and the things that actually delay a wave, which are almost never the engineering.

The four waves

Wave 1 · Prove it — Oracle RMS

The reference data everything else joins to: product, store, supplier, price. Builds `dim_product` and `dim_store` first, so every later wave has something to attach to. **Proves:** the engine works, end to end.

Wave 2 · The volume — SIM and XStore

The two big transactional sources. XStore is the giant and carries the receipt logic (D11). This is where parallelism, cost and the hard-delete question all get answered for real rather than in principle. **Proves:** it scales, and it reconciles.

Wave 3 · The APIs — Epsilon and MoEngage

A different connector class: rate limits, cursor paging, no bulk export to fall back on. **Epsilon carries PII, so this wave cannot start until classification and sign-off are done.** **Proves:** governance actually works.

Wave 4 · The rest — Magento and footfall

Small volumes, third connector class, unreliable arrival. Deliberately last: by now the landing conventions, alarms and idempotency are proven, so late and duplicate files are harmless. **Proves:** the ninth source is cheap.

Why hardest first

Because you want to be wrong **while there is still time to be wrong**.

Doing the easy sources first produces a comfortable burn-up chart and postpones every genuine risk to the end, where there is no slack. Wave 2 is where the architecture either holds or does not; find that out in month two, not month six.

The easy ones at the end are the reward, not the warm-up.

What blocks waves — and is not ours to fix

BLOCKER	BLOCKS	START IT
Network path to on-prem Oracle	waves 1–2	week one (D18)
PII classification and legal sign-off	wave 3	week one
SaaS API credentials and quota	wave 3	week one
Source owners agreeing a contract	everything	week one

All four are other people's calendars. Track them as first-class programme items with named owners on the client side, and report on them weekly — not as a risk register entry nobody reads.

The test at the end of wave 1

Before wave 2 starts, onboard the **second table** in RMS and count the lines of new Python. If it is not close to zero, the seam is wrong (D16) — fix it now, at two tables, not at ninety.

M04 · Delivery Waves And What Blocks Them

Checklist

- ☐ Waves are ordered hardest-first, with a reason
- ☐ Wave 1 builds the conformed dimensions
- ☐ Wave 3 is gated on PII sign-off, and everyone knows it
- ☐ All four blockers have named client-side owners

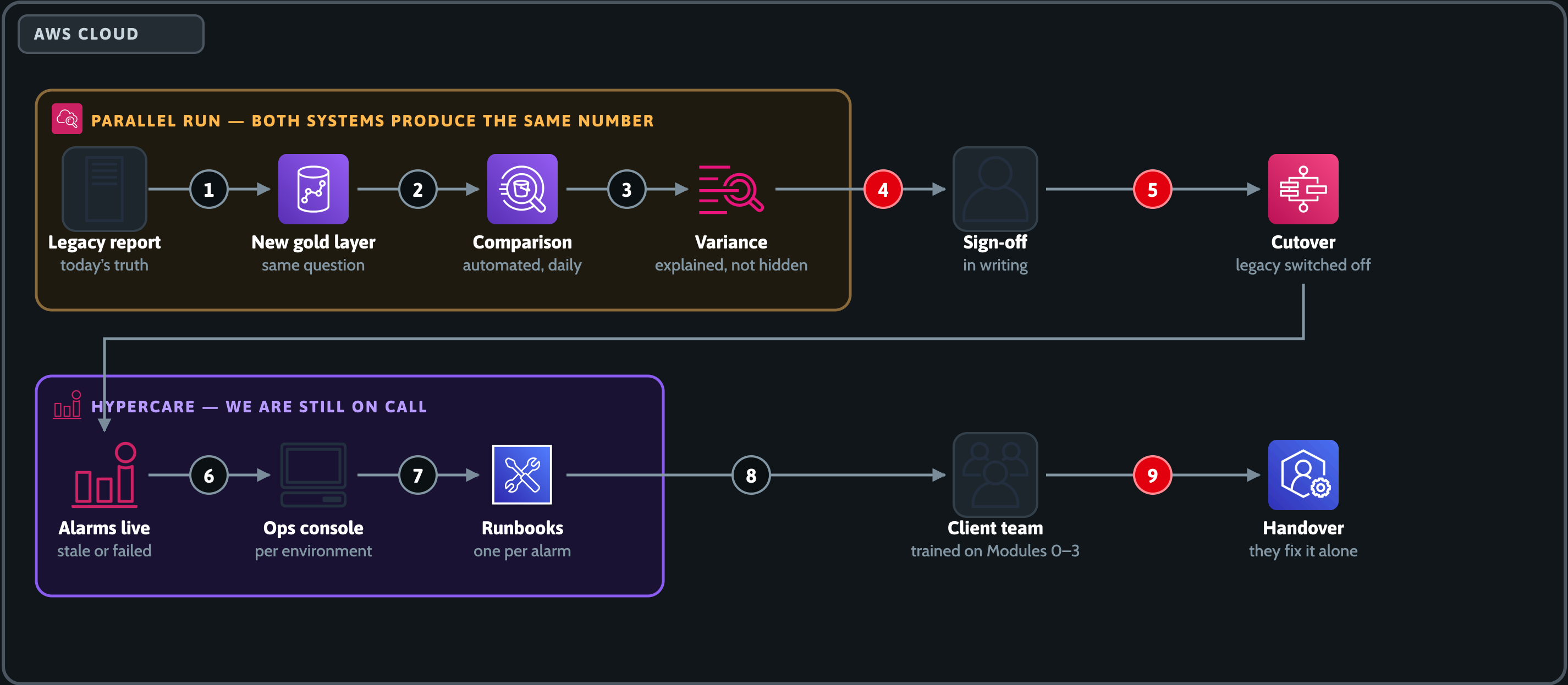
- ☐ The "second table costs zero code" test is run at the end of wave 1
- ☐ Blockers are reported weekly, not buried in a risk log

You've got it when you can...

...be asked why the smallest source is scheduled last and give the real answer — that the schedule is ordered by risk, not by effort.

Go-Live And Operations

Nine steps from parallel run to handover. The project is not finished when it works — it is finished when they can fix it without us.



IN PLAIN ENGLISH

Run both tills side by side until they agree, in writing. Then switch one off — and stay in the shop until the staff can open it without you.

M05 · Go-Live And Operations

Diagram: [_render/M05-go-live-operations.html](#)

What this pattern is for

Getting from "the platform works" to "the client team runs it".

The project is not finished when it works. It is finished when they can fix it without us.

The nine steps

- 1 · **The legacy report keeps running.** It is still the source of truth on day one, and it stays that way until step 5. Nothing is switched off on a promise.
- 2 · **The new gold layer answers the same question.** Same question, new path. Not a better question, not a redesigned metric — the *same* one, so the comparison in step 3 means something.
- 3 · **Compare them daily, automatically.** ★ Not a spreadsheet somebody maintains. A scheduled job that produces a variance figure every morning, because a manual comparison stops happening in week two, exactly when it starts to matter.
- 4 · **Every variance is explained, not hidden.** ★ Some will be legitimate — a rounding rule you deliberately corrected, a bug in the legacy report. **Write each one down.** An unexplained variance is a blocker; a documented one is a decision. This is where credibility is won or lost.
- 5 · **Business sign-off, in writing.** ★ Not "the numbers look right in the meeting". A named person accepting a documented variance list. This is the gate between QA and production (D20), and it is the one people are tempted to soften.

6 · **Cutover.** The legacy report is switched off. If it stays on "just in case", you now maintain two systems forever and nobody trusts either.

7 · **Alarms go live.** Freshness **and** failure (D26). The freshness alarm is the one that matters — a job that silently stopped is far more dangerous than one that failed loudly.

8 · **The ops console and runbooks.** One console per environment. **One runbook entry per alarm** — what it means, what to check, how to re-run safely. An alarm without a runbook costs the responder twenty minutes of orientation every time.

9 · **Handover.** ★ The client team has been through Modules 0–3. The real test: **they handle a nightly failure alone, while we watch.** Not a tabletop exercise — an actual incident, or a deliberately injected one.

Hypercare has an exit criterion

It is not a duration. It is step 9: the client team resolving a real failure without us intervening. Hypercare defined as "four weeks" simply ends when the money does, whether or not anyone is ready.

The variance conversation

Step 4 is where projects like this are actually judged. Two rules:

- **Bring variances to them before they find them.** A variance you disclose is diligence; the same variance discovered by the client is a defect.
- **Never explain a variance as "the old report was wrong" without evidence.** Sometimes it is. Prove it with the source data, and get that agreed too.

M05 · Go-Live And Operations

Checklist

- ☐ Parallel run compares automatically, daily
- ☐ Every variance is documented and explained
- ☐ Sign-off is written, by a named person
- ☐ Legacy is actually switched off at cutover
- ☐ Freshness alarms as well as failure alarms

- ☐ One runbook entry per alarm
- ☐ Ops console live, per environment
- ☐ The client team has resolved a real failure unaided

You've got it when you can...

...define "done" for this programme as something the client team demonstrates rather than something we deliver — and hold that line when the date gets tight.